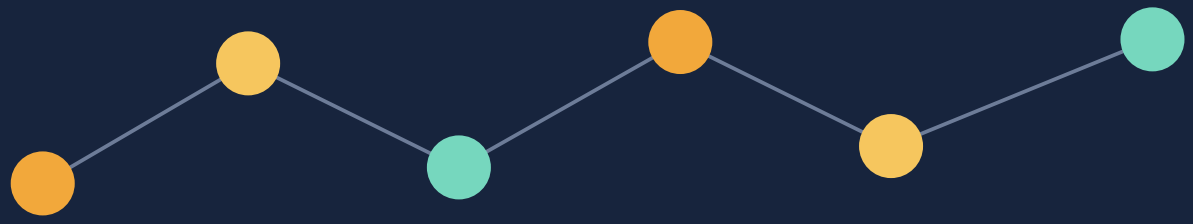
GATE 2027 / COMPLETE SYLLABUS EDITION

Programming Data Structures

Start from the basics. Build every syllabus idea. Then practise.

Section 4: Programming and Data Structures



LEARN → SEE → WORK THROUGH → RETRIEVE → PRACTISE

1 Understand

Each chapter starts in plain language.

2 Visualize

Follow the flow and redraw it.

3 Work

Trace the worked example with pencil.

4 Recall

Answer before checking the solution.

A beginner learning guide + illustrated recall/practice + your supplied reference material

2027 syllabus coverage map

Use this as a route map: every item below has a beginner lesson in this PDF. The supplied handbook/chapter PDFs are gathered in the reference section after the learning and practice pages.

- [ ] **C foundations: values, types, expressions and control**

  Types and conversion • Expressions • Control flow • Arrays and pointers

- [ ] **Recursion and the call stack**

  Base case • Progress • Call stack • Complexity

- [ ] **Arrays and strings**

  Indexing • 2D arrays • C strings • Trade-offs

- [ ] **Stacks, queues and linked lists**

  Stack • Queue • Linked list • Pointer update

- [ ] **Trees, binary search trees and heaps**

  Tree language • Traversals • BST • Binary heap

- [ ] **Graphs and representations**

  Adjacency matrix • Adjacency list • Breadth-first search • Depth-first search

- [ ] **C pointers, arrays and variable lifetime**

  Pointer step • Array indexing • Scope and lifetime • Sequence points

- [ ] **Tree shape, heaps and traversal invariants**

  Full binary tree • Array heap indices • Heap leaves • BST order

Your route through this subject

Follow the lessons in order. Tick a topic only after you can explain its picture without looking; a tick means “I can teach it,” not “I have seen it.”

1. C foundations: values, types, expressions and control
2. Recursion and the call stack
3. Arrays and strings
4. Stacks, queues and linked lists
5. Trees, binary search trees and heaps
6. Graphs and representations
7. C pointers, arrays and variable lifetime
8. Tree shape, heaps and traversal invariants

THE 20-MINUTE LEARNING LOOP

5 min	5 min	7 min	3 min
Read	Redraw	Solve	Recall

IF THIS SUBJECT FEELS COMPLETELY NEW

That is okay. Start with the first lesson, keep a pencil beside you, and redraw its picture once. Attempt the recall prompt before opening the answer guide.

C foundations: values, types, expressions and control

START WITH THE IDEA

C executes instructions that manipulate typed values in memory. To reason about code, track each variable, its type, its storage and the control path.

A CONCRETE FIRST EXAMPLE

If sizeof(int)=4 and an array begins at address 1000, A[5] is at 1000+5*4=1020 (zero-based indexing).

PICTURE IT



IMAGINE IT THIS WAY

Think of memory as numbered lockers: a pointer stores a locker number, while a value is what the locker contains.

THE IDEAS, ONE BY ONE

Types and conversion

Integer, character and floating types have different ranges and arithmetic rules. Conversions may truncate or change signedness.

Expressions

Precedence determines grouping; sequencing rules determine whether a value is safe to read or modify.

Control flow

if/else chooses a path; loops repeat; functions package behavior and communicate through arguments and return values.

Arrays and pointers

An array holds adjacent elements. A pointer stores an address; dereferencing accesses the object at that address.

HOW TO USE IT ON A PROBLEM

Draw the memory or data structure before changing it. Trace each pointer/index operation, preserve invariants, then state time and extra space.

WATCH OUT Never dereference an uninitialized, null or out-of-bounds pointer.

TRY FROM MEMORY

Cover this page. Explain C foundations: values, types, expressions and control using the picture, then give one example before looking again.

Recursion and the call stack

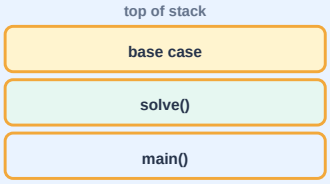
START WITH THE IDEA

A recursive function solves a smaller instance of itself. Every call has its own parameters, locals and return point, stored until that call finishes.

A CONCRETE FIRST EXAMPLE

For factorial(3): 3·factorial(2), then 2·factorial(1), then base case 1; returns unwind to 2 then 6.

PICTURE IT



IMAGINE IT THIS WAY

Each call places a sticky note on the stack; return peels off the newest note.

THE IDEAS, ONE BY ONE

Base case
The stopping condition must return without making another recursive call.

Progress
Each recursive step must move toward a base case.

Call stack
Calls are pushed in order and return in reverse order; this is LIFO.

Complexity
Count calls and work per call; stack space often equals maximum recursion depth.

HOW TO USE IT ON A PROBLEM

Draw the memory or data structure before changing it. Trace each pointer/index operation, preserve invariants, then state time and extra space.

WATCH OUT A correct answer may still use too much time or stack space.

TRY FROM MEMORY

Cover this page. Explain Recursion and the call stack using the picture, then give one example before looking again.

Arrays and strings

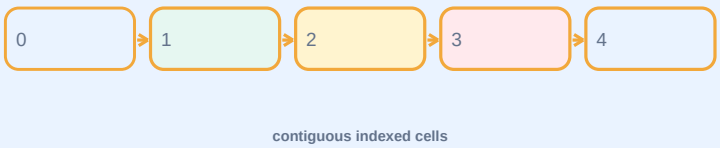
START WITH THE IDEA

Arrays give direct access by index because elements occupy adjacent memory. Inserting inside usually requires shifting; strings in C are character arrays terminated by a null byte.

A CONCRETE FIRST EXAMPLE

A 3*4 int matrix with 4-byte ints stores M[2][1] at base+(2*4+1)*4 = base+36.

PICTURE IT



contiguous indexed cells

IMAGINE IT THIS WAY

An array is a numbered shelf: direct reach is fast, but making space in the middle moves boxes.

THE IDEAS, ONE BY ONE

Indexing

Address = base + index*element size, for a valid zero-based index.

2D arrays

C stores arrays in row-major order; later columns of a row are adjacent.

C strings

The visible characters are followed by \0. Space must include this terminator.

Trade-offs

Array reads by index are O(1); search is O(n) without ordering; middle insertion/deletion shifts values.

HOW TO USE IT ON A PROBLEM

Draw the memory or data structure before changing it. Trace each pointer/index operation, preserve invariants, then state time and extra space.

WATCH OUT Reserve one extra byte for a C string's null terminator.

TRY FROM MEMORY

Cover this page. Explain Arrays and strings using the picture, then give one example before looking again.

Stacks, queues and linked lists

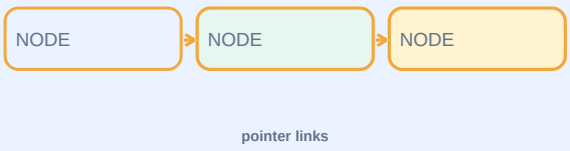
START WITH THE IDEA

These structures differ by how they connect items and which item is removed first. Their operations model common tasks such as parsing, scheduling and traversal.

A CONCRETE FIRST EXAMPLE

Stack: push 4, push 7, pop gives 7. Queue: enqueue 4, enqueue 7, dequeue gives 4.

PICTURE IT



IMAGINE IT THIS WAY

Stack is a plate pile; queue is a ticket line; linked list is a chain of clues.

THE IDEAS, ONE BY ONE

Stack

Last in, first out. Push/pop at one end; used for recursion, parentheses and postfix evaluation.

Queue

First in, first out. Enqueue at rear and dequeue at front; circular arrays reuse slots by modulo capacity.

Linked list

Nodes hold data and links. Insert/delete can be O(1) when the location is known; search remains O(n).

Pointer update

Save the old next link before overwriting it, or the remainder of the list may be lost.

HOW TO USE IT ON A PROBLEM

Draw the memory or data structure before changing it. Trace each pointer/index operation, preserve invariants, then state time and extra space.

WATCH OUT In a circular queue, state the convention used to distinguish empty and full.

TRY FROM MEMORY

Cover this page. Explain Stacks, queues and linked lists using the picture, then give one example before looking again.

Trees, binary search trees and heaps

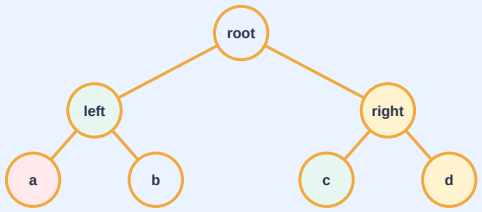
START WITH THE IDEA

Trees organize hierarchical data. A BST orders keys to support search; a binary heap keeps only a parent-child priority rule for efficient min/max removal.

A CONCRETE FIRST EXAMPLE

Insert 8,3,10 into a BST: inorder is 3,8,10. A max-heap places its largest value at the root but does not sort siblings.

PICTURE IT



IMAGINE IT THIS WAY

A BST is an ordered library; a heap is a podium that exposes the current winner.

THE IDEAS, ONE BY ONE

Tree language

Root, parent, child, leaf, depth and height describe structure. A tree with n nodes has n-1 edges.

Traversals

Preorder is root-left-right; inorder left-root-right; postorder left-right-root; level order uses a queue.

BST

Left subtree keys are smaller and right subtree keys larger, according to the chosen duplicate policy. Inorder is sorted.

Binary heap

Complete binary tree with parent priority over children. In a 0-based array, children of i are 2i+1 and 2i+2.

HOW TO USE IT ON A PROBLEM

Draw the memory or data structure before changing it. Trace each pointer/index operation, preserve invariants, then state time and extra space.

WATCH OUT Heap order does not imply the entire array is sorted.

TRY FROM MEMORY

Cover this page. Explain Trees, binary search trees and heaps using the picture, then give one example before looking again.

Graphs and representations

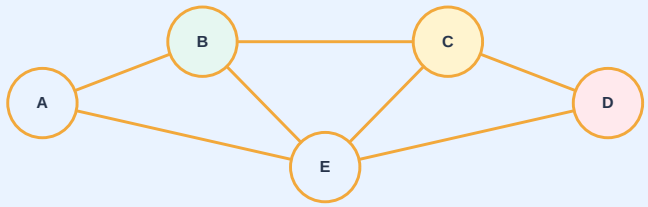
START WITH THE IDEA

A graph consists of vertices and edges. It may be directed/undirected and weighted/unweighted; representation affects memory and operation cost.

A CONCRETE FIRST EXAMPLE

Starting at S, BFS processes all distance-1 neighbors before distance-2 vertices. Mark a vertex visited when enqueuing to avoid duplicates.

PICTURE IT



IMAGINE IT THIS WAY

A city map can be kept as a full road chart (matrix) or each city's neighbor list (adjacency list).

THE IDEAS, ONE BY ONE

Adjacency matrix

V*V table; O(V^2) space and O(1) edge lookup.

Adjacency list

Each vertex stores its neighbors; O(V+E) space and efficient neighbor iteration.

Breadth-first search

BFS uses a queue and explores by distance. In an unweighted graph it finds shortest paths in number of edges.

Depth-first search

DFS uses recursion or a stack to explore deeply and backtrack; useful for reachability and structure discovery.

HOW TO USE IT ON A PROBLEM

Draw the memory or data structure before changing it. Trace each pointer/index operation, preserve invariants, then state time and extra space.

WATCH OUT BFS shortest paths do not apply to arbitrary weighted edges.

TRY FROM MEMORY

Cover this page. Explain Graphs and representations using the picture, then give one example before looking again.

C pointers, arrays and variable lifetime

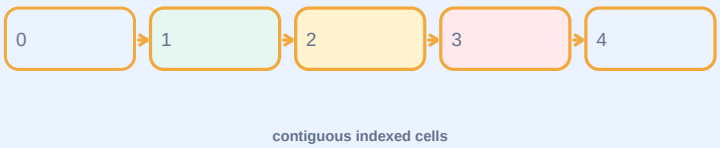
START WITH THE IDEA

A pointer stores an address. Pointer arithmetic moves by whole pointed-to objects; arrays are contiguous, but the array name is not an assignable pointer variable.

A CONCRETE FIRST EXAMPLE

With int a[3]={10,20,30}; int *p=a; p++; *p=25; the array becomes {10,25,30}.

PICTURE IT



contiguous indexed cells

IMAGINE IT THIS WAY

An array is a row of labeled boxes; a pointer is the address written on a movable bookmark.

THE IDEAS, ONE BY ONE

Pointer step

If p points to an int, p+1 advances by sizeof(int) bytes. *p reads or writes the pointed-to int.

Array indexing

a[i] is equivalent to *(a+i). Keep the address and stored value separate while tracing.

Scope and lifetime

A block-local variable is visible inside its block. A static local keeps its value between calls; automatic locals are recreated per call.

Sequence points

When a C expression modifies an object more than once without a required ordering, its behavior may be undefined. Do not invent an evaluation order.

HOW TO USE IT ON A PROBLEM

Draw the memory or data structure before changing it. Trace each pointer/index operation, preserve invariants, then state time and extra space.

WATCH OUT Incrementing p changes the pointer; incrementing *p changes the data.

TRY FROM MEMORY

Cover this page. Explain C pointers, arrays and variable lifetime using the picture, then give one example before looking again.

Tree shape, heaps and traversal invariants

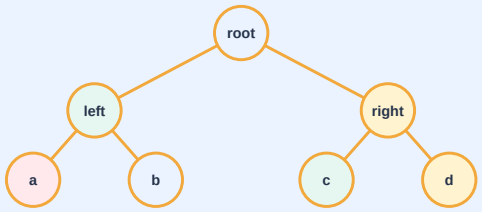
START WITH THE IDEA

Trees are recursive structures. Binary search trees preserve key order; binary heaps preserve parent-child priority and a complete-tree shape.

A CONCRETE FIRST EXAMPLE

A 1-based heap with n=9 has leaves at indices 5, 6, 7, 8 and 9: floor(9/2)+1 through 9.

PICTURE IT



IMAGINE IT THIS WAY

A heap is a pyramid stored row by row. A BST is a set of signposts that route smaller keys left and larger keys right.

THE IDEAS, ONE BY ONE

Full binary tree

Every node has either zero or two children. If it has i internal nodes, it has i+1 leaves and 2i+1 total nodes.

Array heap indices

With 1-based indexing, children of i are 2i and 2i+1; parent is floor(i/2). With 0-based indexing, the formulas change.

Heap leaves

In a complete heap of n items indexed from 1, leaves occupy indices floor(n/2)+1 through n.

BST order

Every left-subtree key is smaller and every right-subtree key is larger (for distinct keys). Inorder traversal is sorted.

HOW TO USE IT ON A PROBLEM

Draw the memory or data structure before changing it. Trace each pointer/index operation, preserve invariants, then state time and extra space.

WATCH OUT State the indexing convention before applying heap formulas.

TRY FROM MEMORY

Cover this page. Explain Tree shape, heaps and traversal invariants using the picture, then give one example before looking again.

Past-paper trail and coverage check

WHAT THE PAPERS MATCH

Verified anchors: 2018 Q20-Q21/Q29/Q46 (trees, C recursion/pointers and heaps); 2022 Q15/Q21/Q28/Q43/Q62 (lists, C, BSTs and stack/queue); 2023 Q11-Q12/Q34/Q45-Q46/Q58; 2024 CS1 Q18-Q19/Q41/Q43; 2025 CS1 Q26/Q34-Q35/Q62 and CS2 Q13/Q19/Q33/Q35/Q45; 2026 CS1 Q23/Q33-Q34/Q39-Q40/Q62 and CS2 Q12/Q49-Q50/Q60-Q61.

WHAT THIS EDITION ADDS

Added C pointer arithmetic, array/pointer equivalence, static lifetime, undefined sequencing warning, full binary tree identities, heap index formulas and BST inorder invariant.

SYLLABUS BOUNDARY

The 2027 list specifies C and core structures; language features outside C and implementation-specific compiler behavior are not core.

HOW TO USE THIS CHECK

A question is a match when its tested skill is taught in the lesson pages, even if the story or numbers are new. Anchors are selected verified examples, not an exhaustive list; the separate Coverage and Gap Audit PDF identifies years whose scans could not be checked question by question.

Close the notes: try before you peek

For each card, explain the idea from memory and recreate the lesson example on paper.

1. **C foundations: values, types, expressions and control**

   Without notes: define Arrays and pointers and explain why this example works. Then recreate the steps on the lesson page.

2. **Recursion and the call stack**

   Without notes: define Complexity and explain why this example works. Then recreate the steps on the lesson page.

3. **Arrays and strings**

   Without notes: define Trade-offs and explain why this example works. Then recreate the steps on the lesson page.

4. **Stacks, queues and linked lists**

   Without notes: define Pointer update and explain why this example works. Then recreate the steps on the lesson page.

5. **Trees, binary search trees and heaps**

   Without notes: define Binary heap and explain why this example works. Then recreate the steps on the lesson page.

6. **Graphs and representations**

   Without notes: define Depth-first search and explain why this example works. Then recreate the steps on the lesson page.

Close the notes: try before you peek

For each card, explain the idea from memory and recreate the lesson example on paper.

7

C pointers, arrays and variable lifetime

Without notes: define Sequence points and explain why this example works. Then recreate the steps on the lesson page.

8

Tree shape, heaps and traversal invariants

Without notes: define BST order and explain why this example works. Then recreate the steps on the lesson page.

Compare with the worked examples

Use this only after attempting the recall prompts. The full reasoning is on the numbered lesson page.

1. **LESSON 1: C foundations: values, types, expressions and control**

   If sizeof(int)=4 and an array begins at address 1000, A[5] is at 1000+5*4=1020 (zero-based indexing).

2. **LESSON 2: Recursion and the call stack**

   For factorial(3): 3·factorial(2), then 2·factorial(1), then base case 1; returns unwind to 2 then 6.

3. **LESSON 3: Arrays and strings**

   A 3*4 int matrix with 4-byte ints stores M[2][1] at base+(2*4+1)*4 = base+36.

4. **LESSON 4: Stacks, queues and linked lists**

   Stack: push 4, push 7, pop gives 7. Queue: enqueue 4, enqueue 7, dequeue gives 4.

5. **LESSON 5: Trees, binary search trees and heaps**

   Insert 8,3,10 into a BST: inorder is 3,8,10. A max-heap places its largest value at the root but does not sort siblings.

6. **LESSON 6: Graphs and representations**

   Starting at S, BFS processes all distance-1 neighbors before distance-2 vertices. Mark a vertex visited when enqueuing to avoid duplicates.

Compare with the worked examples

Use this only after attempting the recall prompts. The full reasoning is on the numbered lesson page.

7. LESSON 7: C pointers, arrays and variable lifetime

With int a[3]={10,20,30}; int *p=a; p++; *p=25; the array becomes {10,25,30}.

8. LESSON 8: Tree shape, heaps and traversal invariants

A 1-based heap with n=9 has leaves at indices 5, 6, 7, 8 and 9: floor(9/2)+1 through 9.

Your practice and reference library

01 **Beginner lessons**

Plain-language explanation, visual sequence, example and watch-outs for every listed 2027 topic cluster.

02 **Recall and GATE-style practice**

The illustrated recap section includes questions and a separate answer explanation.

03 **Detailed reference**

Your supplied subject handbook and chapter PDFs follow for extra depth, reading and worked material.

Included source documents

Illustrated recall and GATE-style practice	PDS - Programming and Data Structures - GATE 2027 Illustrated Notes.pdf	12 pages
Detailed supplied reference	DS&P.pdf	44 pages
Detailed supplied reference	GATE2027_Chapter1_C_Programming.pdf	11 pages
Detailed supplied reference	GATE2027_Chapter2_DSA1_Arrays_Strings_Recursion.pdf	5 pages
Detailed supplied reference	GATE2027_Chapter3_DSA2_LinkedLists_Stacks_Queues.pdf	5 pages
Detailed supplied reference	GATE2027_Chapter4_DSA3_Trees_BST_Heaps.pdf	4 pages
Detailed supplied reference	GATE2027_Chapter5_DSA4_Graphs.pdf	5 pages

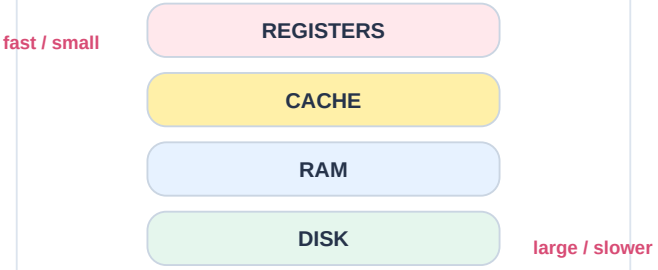
The official 2027 syllabus map precedes the lessons. Original reference pages are retained so you can use them for slower reading and extra practice.

Programming and Data Structures

GATE 2027 | VISUAL LEARNING NOTES | STARTING FROM THE BASICS

Trace the state. Draw the links. Count the work.

PICTURE IT



HOW TO USE THIS NOTEBOOK

1. **Learn the idea**
   Read the plain-language explanation before memorizing a formula.
2. **See it**
   Study the labeled picture and redraw it from memory.
3. **Follow a solve**
   Work through the examples step by step.
4. **Test yourself**
   Attempt the GATE-style questions before opening the key.

THE PATH YOU WILL FOLLOW

- Programming
- Linear and linked structures
- Trees and graphs

IF YOU FEEL COMPLETELY NEW

That is okay. Start with the first topic page, keep a pencil beside you, and copy each diagram once. Do not jump straight to the answer key. The questions are meant to come after the concepts.

Your route through this subject

Follow the topics in order. Each page teaches one idea, shows a picture, gives a recall cue and points out a common trap.

PROGRAMMING

- [ ] Programming in C.
- [ ] Recursion.

LINEAR AND LINKED STRUCTURES

- [ ] Arrays, stacks and queues.
- [ ] Linked lists.

TREES AND GRAPHS

- [ ] Trees and binary search trees.
- [ ] Binary heaps.
- [ ] Graphs.

THE 20-MINUTE LEARNING LOOP

5 min	5 min	7 min	3 min
Read	Redraw	Solve	Recall

Tick a topic only after you can explain its picture without looking. A tick means "I can teach it," not "I have seen it."

BEFORE EACH STUDY SESSION

Pick one topic. Read its idea, redraw its diagram, follow the worked examples, then answer questions with the notes closed. Circle only the step where you got stuck - that is your next revision target.

CONCEPT 01 / PDS

C: values, addresses and arrays

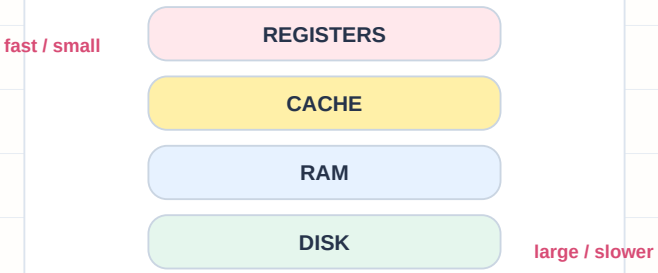
START WITH THE IDEA

A variable stores a value; a pointer stores an address. In C, an array name often converts to a pointer to its first element, but an array is not a pointer variable. Indexing uses contiguous elements and a known element size.

A CONCRETE FIRST EXAMPLE

C stores values in memory; a pointer stores an address. An array occupies adjacent locations, while a pointer variable can be changed to point elsewhere. The type tells pointer arithmetic how many bytes one step moves. If int is 4 bytes and p points to address 1000, p+1 points to 1004. Dereferencing p reads the int stored there.

PICTURE IT



IMAGINE IT THIS WAY

Think of memory as numbered lockers: a pointer stores a locker number, and an array is a row of lockers.

HOW TO USE IT ON A PROBLEM

Draw addresses and values separately; evaluate pointer expressions one step at a time.

WATCH OUT

Never dereference an uninitialized, null or out-of-bounds pointer.

TRY FROM MEMORY

Cover the page and explain C: values, addresses and arrays using the diagram. Give one example before looking again.

CONCEPT 02 / PDS

Recursion and call stack

START WITH THE IDEA

A recursive function needs a base case and progress toward it. Each call keeps parameters, locals and a return point in an activation record. Time may be linear, logarithmic or exponential depending on the recurrence.

A CONCRETE FIRST EXAMPLE

A function call pauses its caller and places parameters, local variables and a return address in an activation record. Recursion works when each call makes progress toward a base case. For factorial(3), calls go 3 -> 2 -> 1; factorial(1) returns 1, then the pending calls return 2 and 6.

PICTURE IT



LIFO: top leaves first

IMAGINE IT THIS WAY

Each call is a new sticky note on a stack; returns peel notes off in reverse order.

HOW TO USE IT ON A PROBLEM

Expand the first few calls, identify base case, then write T(n).

WATCH OUT

A correct return value does not imply acceptable stack usage.

TRY FROM MEMORY

Cover the page and explain Recursion and call stack using the diagram. Give one example before looking again.

CONCEPT 03 / PDS

Arrays and strings

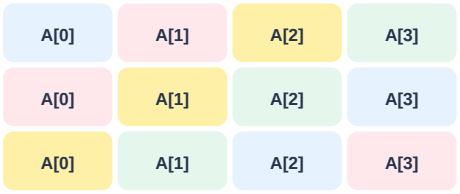
START WITH THE IDEA

Array indexing is constant-time with a valid index; insertion in the middle shifts elements. C strings are character arrays ending with the null byte \0. Row-major 2D arrays store each row contiguously.

A CONCRETE FIRST EXAMPLE

Arrays support direct indexing because the address is base+index x element-size. Insertions in the middle may shift many values. C strings need a terminating null character, so their storage needs one extra byte. If an int array starts at 2000 and each int is 4 bytes, A[5] is at 2000+5 x 4=2020.

PICTURE IT



index -> element

IMAGINE IT THIS WAY

An array is a numbered shelf; reaching shelf 40 is direct, but inserting a new item may shift many boxes.

HOW TO USE IT ON A PROBLEM

For a 2D array, compute row offset before column offset.

WATCH OUT

Do not count the null terminator as visible string content, but reserve space for it.

TRY FROM MEMORY

Cover the page and explain Arrays and strings using the diagram. Give one example before looking again.

CONCEPT 04 / PDS

Linked lists

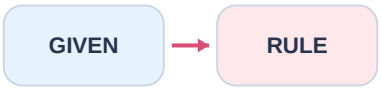
START WITH THE IDEA

A node stores data and one or more links. Insertion/deletion can be O(1) once the location is known, but searching is O(n). Singly linked lists move forward; doubly linked lists also keep a previous link.

A CONCRETE FIRST EXAMPLE

A linked list stores each value with a link to another node. You can insert in constant time once you have the correct node, but finding that node still takes a traversal. To insert X after A: save A.next, set X.next to the saved link, then set A.next=X.

PICTURE IT



work through one example carefully

IMAGINE IT THIS WAY

A treasure hunt where each clue points to the next clue.

HOW TO USE IT ON A PROBLEM

Before changing links, save the old next pointer.

WATCH OUT

A missing link can lose the rest of the list; a cycle can make traversal never end.

TRY FROM MEMORY

Cover the page and explain Linked lists using the diagram. Give one example before looking again.

CONCEPT 05 / PDS

Stacks and queues

START WITH THE IDEA

A stack is LIFO; a queue is FIFO. Circular arrays reuse freed positions with modular arithmetic. Stacks model function calls and parsing; queues model waiting lines and BFS frontiers.

A CONCRETE FIRST EXAMPLE

A stack removes the newest item first (LIFO); a queue removes the oldest item first (FIFO). Circular queues wrap indices with modulo capacity so freed slots can be reused. Push 4, then 7: pop returns 7. Enqueue 4, then 7: dequeue returns 4.

PICTURE IT



FIFO: first arrival leaves first

IMAGINE IT THIS WAY

Stack: plates. Queue: ticket counter.

HOW TO USE IT ON A PROBLEM

Write the structure after every push/pop or enqueue/dequeue.

WATCH OUT

In a circular queue, distinguish empty and full using the chosen convention.

TRY FROM MEMORY

Cover the page and explain Stacks and queues using the diagram. Give one example before looking again.

CONCEPT 06 / PDS

Trees, BSTs and heaps

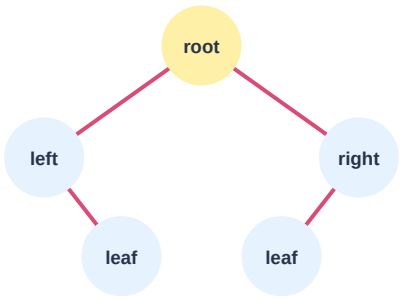
START WITH THE IDEA

A tree has a root and subtrees. BST keys obey left < node < right, so inorder traversal is sorted. A binary heap is a complete tree with parent priority over children; it is not a search tree.

A CONCRETE FIRST EXAMPLE

A binary search tree orders keys left < node < right, so inorder traversal is sorted. A heap is a complete tree whose parent dominates its children; it is designed to expose the minimum or maximum quickly. Insert 8, 3, 10 into a BST: inorder is 3,8,10. In a max-heap, 10 is at the root, but the remaining keys need not be sorted.

PICTURE IT



IMAGINE IT THIS WAY

BST is an organized library; a heap is a podium that keeps only the current winner at the top.

HOW TO USE IT ON A PROBLEM

For a heap stored from index 0: children of i are 2i+1 and 2i+2.

WATCH OUT

A heap guarantees parent/child order, not sorted order across siblings.

TRY FROM MEMORY

Cover the page and explain Trees, BSTs and heaps using the diagram. Give one example before looking again.

CONCEPT 07 / PDS

Graphs and representations

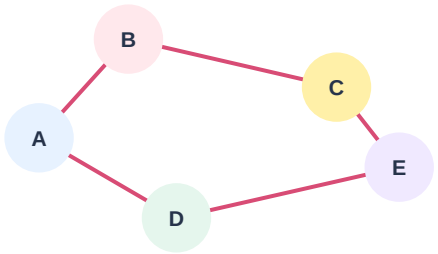
START WITH THE IDEA

Adjacency lists use O(V+E) space; adjacency matrices use O(V^2) and give O(1) edge lookup. BFS explores by distance in unweighted graphs; DFS follows a path and backtracks.

A CONCRETE FIRST EXAMPLE

A graph can be stored as neighbor lists or an adjacency matrix. BFS uses a queue and finds shortest paths when edges are unweighted; DFS uses a stack or recursion to explore deeply before backtracking. In an unweighted graph, BFS from S visits all distance-1 neighbors before distance-2 nodes.

PICTURE IT



vertices + edges

IMAGINE IT THIS WAY

A graph is a city map; adjacency list is each city's neighbor list, matrix is the full road chart.

HOW TO USE IT ON A PROBLEM

Mark visited when enqueuing in BFS to avoid duplicates.

WATCH OUT

BFS shortest paths apply to unweighted edges, not arbitrary weights.

TRY FROM MEMORY

Cover the page and explain Graphs and representations using the diagram. Give one example before looking again.

Let's solve it together

Cover the answer with your hand. Try each step first, then compare your reasoning.

EXAMPLE 1: Recursive sum

QUESTION

What does sum([2,4,6],3) return if sum(a,n)=0 for n=0, else a[n-1]+sum(a,n-1)?

WORK THROUGH THE STEPS

sum(a,3)=6+sum(a,2); sum(a,2)=4+sum(a,1); sum(a,1)=2+sum(a,0); sum(a,0)=0. There are n

ANSWER 12; time O(n), stack O(n).

Check: write the key rule beside your answer. If it does not fit the rule, redo the model before recalculating.

EXAMPLE 2: BST traversal

QUESTION

Insert 8, 3, 10, 1, 6 into an empty BST. Give the inorder sequence.

WORK THROUGH THE STEPS

The BST places 3 left of 8, 10 right, 1 left of 3 and

ANSWER 1, 3, 6, 8, 10.

Check: write the key rule beside your answer. If it does not fit the rule, redo the model before recalculating.

Now you try

Attempt these without the notes. The question types are marked MCQ, MSQ or NAT. Answers and explanations are on the next page.

MCQ A stack receives push(1), push(2), push(3), pop(), push(4). What is the top?

(A) 1 (B) 2 (C) 3 (D) 4

NAT A C int array begins at byte address 1000. If sizeof(int)=4, what is the address of A[5]?

Assume zero-based indexing.

MCQ Which traversal of a BST returns keys in sorted order?

(A) Preorder (B) Inorder (C) Postorder (D) Level order

MSQ Which are true?

(A) A binary heap is always a BST (B) BFS uses a queue (C) DFS can use a stack (D) Adjacency matrix space is O(V^2)

MCQ Which structure is most directly used to evaluate a postfix expression?

(A) Queue (B) Stack (C) Heap (D) Adjacency matrix

Exam habit: decide what is being asked before doing arithmetic. Show units, assumptions and one line of reasoning.

Answers with the why

Read the explanation even when your answer is correct. The goal is to learn the reasoning pattern.

1. D

WHY

The pop removes 3; push(4) makes 4 the new top.

2. 1020

WHY

Address = base + index x size = 1000 + 5x4.

3. B

WHY

BST ordering makes left-root-right (inorder) sorted.

4. B, C, D

WHY

Heap order is not BST search order; the other statements are standard.

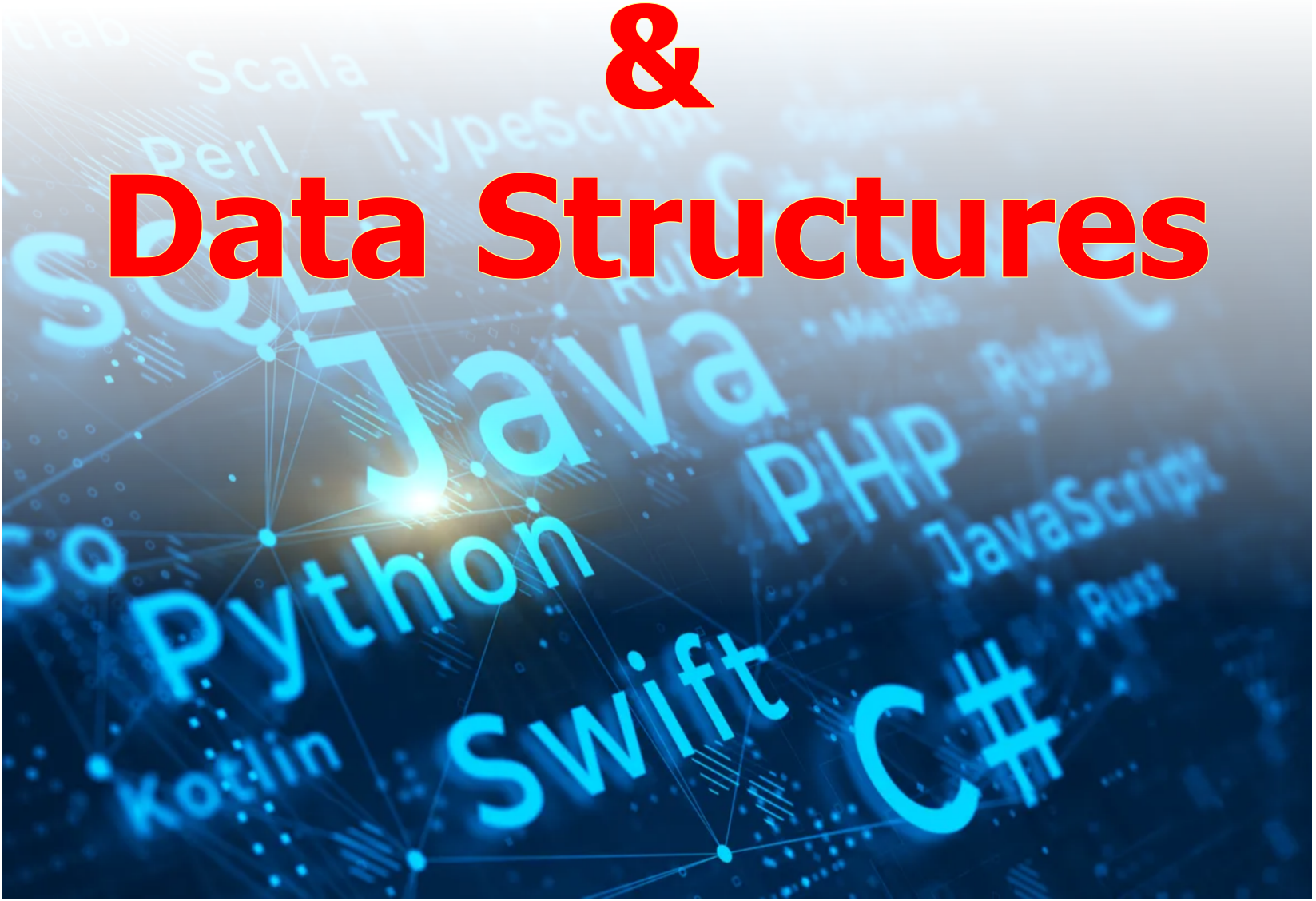
5. B

WHY

Push operands; when an operator appears, pop operands and push the result.

Review loop: mark each missed concept page, redraw its picture, and retry a similar problem tomorrow.

Programming & Data Structures



Published By:

Physics Wallah

ISBN: 978-93-94342-39-2

Mobile App: Physics Wallah (Available on Play Store)

Website: www.pw.live

Email: support@pw.live

Rights

All rights will be reserved by Publisher. No part of this book may be used or reproduced in any manner whatsoever without the written permission from author or publisher.

In the interest of student's community:

Circulation of soft copy of Book(s) in PDF or other equivalent format(s) through any social media channels, emails, etc. or any other channels through mobiles, laptops or desktop is a criminal offence. Anybody circulating, downloading, storing, soft copy of the book on his device(s) is in breach of Copyright Act. Further Photocopying of this book or any of its material is also illegal. Do not download or forward in case you come across any such soft copy material.

Disclaimer

A team of PW experts and faculties with an understanding of the subject has worked hard for the books.

While the author and publisher have used their best efforts in preparing these books. The content has been checked for accuracy. As the book is intended for educational purposes, the author shall not be responsible for any errors contained in the book.

The publication is designed to provide accurate and authoritative information with regard to the subject matter covered.

This book and the individual contribution contained in it are protected under copyright by the publisher.

(This Module shall only be Used for Educational Purpose.)

Programming & Data Structure

INDEX

1. Data Types and Operators ..... 5.1 – 5.5
2. Control Flow Statements ..... 5.6 – 5.8
3. Storage Class & Function ..... 5.9 – 5.12
4. Arrays and Pointers ..... 5.13 – 5.22
5. Strings ..... 5.23 – 5.26
6. Types of Data Structure, Array & Linked List ..... 5.27 – 5.31
7. Stack and Queue ..... 5.32 – 5.33
8. Tree Data Structure ..... 5.34 – 5.38
9. Graph Traversal 5.39
10. Hashing ..... 5.40 – 5.41

1 DATA TYPES AND OPERATORS

1.1 Data Types

1.1.1 Primitive Data Type

(a) Integer Types:

- ✓ short int, unsigned short int
- ✓ int, unsigned int
- ✓ long int, unsigned long int
- ✓ long long int, unsigned long long int

(b) Character Types:

- ✓ char,unsigned char

(c) Floating Types:

- ✓ float, double, long double

(d) Other:

- ✓ void, bool

1.1.2 Non – Primitive Data Type

(a) Derived data type:

- ✓ Array
- ✓ Pointer
- ✓ String

(b) User defined data type:

- ✓ Structure
- ✓ union
- ✓ Enum
- ✓ typedef

- C standard does not specify how many bits or bytes to be allocated for every type and different compiler may choose different ranges.
- Only restriction is that short and int types are at least 16 bits, long types are at least 32 bits and short is no longer than int which is no longer that long type.

1.2 Operators

Depending upon the number of operands, an operator in C language can be unary, binary or ternary.

1.2.1 Arithmetic Operators

(i) Addition (+)
(ii) Subtraction (-)
(iii) Multiplication (×)
(iv) Division (/)
(v) Modulus (%)

- Both operands for % operator must be integer types, otherwise error will be raised.
- The sign of the result for modulo operator is machine dependent.

1.2.2 Relational Operators

(i) Less than (<)
(ii) Greater than (>)
(iii) Less than equal to (< =)
(iv) Greater than equal to (> =)
(v) Equal checking (= =)
(vi) Not Equal to (! =)

- The result of these operators is always 0 to 1

  Ex.1: printf ("%d",20 > 10)
  O/P: 1
  Ex.2: printf ("%d",20 = = 10)
  O/P: 0

1.2.3 Assignment Operator (=)

- Used to assign a value or value of an expression to a variable.
- Typically, the system is
  Lvalue = Rvalue
- Lvalue must be a variable.
- Lvalue cannot be a literal or expression.
- Rvalue can be an expression, variable, literal.

Ex1.

The following are invalid

```
10 = a;
a + b = c;
```

- The result of assignment statement is the value we are assigning i.e……

```
int x;
x = 3 + (x = 10); is perfectly valid.
Firstly x = 10 evaluated and
(1) 10 is assigned to x   (2) then the result of (x = 10) will be 10
so,
x = 3 + 10
```

x = 13
Finally, 13 is assigned to x.

1.2.4 Logical Operators

(i) Logical AND (&&)
(ii) Logical OR (||)
(iii) Logical NOT (!)

- Just like rational operators, the result of every logical operator is either 0 or 1.
- Logical NOT is a unary operator
- Logical NOT converts a non-zero operand into 0 and a zero operand in 1.

1.2.5 Short Circuiting in Logical Operators

- In case of logical AND, the second operand is evaluated only if the first operand is evaluated to be true.
  If the first operand is evaluated as false then the second operand is not evaluated.
- In case of logical OR operation, the second operand is not evaluated if the first operand is evaluated as true.

Example1:

```
void main () {
printf (“Hello”) || printf (“Pankaj”);
}
```

O/P: Hello

printf function display everything written within double quotes on the monitor and the result / value returned by printf () is the number of characters successfully displayed on the screen.

So, printf (“Hello”) prints Hello and return 5.

So, the expression

```
printf (“Hello”) || printf (“Pankaj”);
```

becomes

```
5 || printf (“Pankaj”)
```

As the first operand for logical OR is evaluated as true, second printf () will never execute.

1.2.6 Increment and Decrement Operators

(i) pre increment ++x
(ii) post increment x++
(iii) pre decrement - -x
(iv) post decrement x - -

- unary operator.
- can’t be applied on constant / literals.
- Pre increment: can be viewed as 2 step operators.
  1st step: Increment the value of variable.
  2nd step: After increment, use the value of variable.
- Post increment: can be viewed as 2 step operators
  1st step: Use the value.
  2nd step: Increase the value of variable by 1.

1.2.7 Bitwise operators

(i) Bitwise AND (&)
(ii) Bitwise OR (|)
(iii) Bitwise XOR (^)
(iv) Bitwise Left Shift (<<)
(v) Bitwise Right Shift (>>)
(vi) Bitwise Not (~)

- All these operators work on binary representation of numbers.
- Typically, faster than arithmetic operators and other operators.

```
#include<stdio.h>
    int main () {
            int x = 3, y = 6;
            printf ("%d\n",x & y);    // output 2
            printf ("%d\n",x|y);      // output 7
            printf ("%d\n", x^y)      // output 5
            return 0;
            }
```

binary representation of 3: 00000000...0011
binary representation of 6: 00000000...0110

3 & 6: 00000000...0010
3 | 6: 00000000...0111
3 ^ 6: 00000000...0101

XOR of two bits is 1 if both bits are different.
XOR of two bits is 0 if both bits are same.

- ~ x has output as – (x + 1)
- printf ("%d\n", ~2) has output -3 i.e., -(2 + 1)

- Comma has lowest precedence among all operators in C language.
- sizeof() is used
  (i) To find the number of elements present in an array.
  (ii) To dynamically allocate block of memory.

1.2.8 Ternary operator (? :)

- It requires 3 operands i.e., Left, Middle, Right
  Left ? Middle : Right

- If left expression evaluates as true, then the value returned is middle argument otherwise the returned value is Right expression

int a;

a = 7 > 2 ? 3 × 2 + 5 : 6! = 7

⇒ Left expression: 7 > 2 is true

⇒ So, the value returned would be the middle argument.

i.e., 3 × 2 + 5 = 11

so, a will get 11.

Note:
(a) No of '?' and ':' should be equal.
(b) Every colon (:) should match with just before '?'.
(c) Every ? followed by : not immediately but following.

Example: int a;

a = 2 > 5 ? 10 : !5! = 2 > 5 ? 20 : 30

L_1 M_1 R_1

2 > 5 is false.

so, for Leftmost ? the value returned is its right expression.

a = ! 5 ! = 2 > 5 ? 20 : 30 :

Left Mid Right

Left expression:

!5! = 2 > 5

0! = 2 > 5

0! =0

0 i.e., false.

As left operand is false, the final value is right expression

a = 30

Operators	Associativity
() [] -> .	Left to right
! ~ ++ -- + - * *(type)* size of	Right to left
* / %	Left to right
+ -	Left to right
<< >>	Left to right
< <= > >=	Left to right
= = ! =	Left to right
&	Left to right
^	Left to right
|	Left to right
&&	Left to right
||	Left to right
? :	Right to left
= += -= *= /= %= &= ^= |= <<= >>=	Right to left
,	Left to right

❐❐❐

2 CONTROL FLOW STATEMENTS

2.1 Switch Statement

- "Case Value" can be of character type and int type.
- There can be one or many number of cases.
- Statements associated with a matching case executes until a break statement is reached.
- The position of default case does not matter. It can be placed anywhere.
- Default case is optional.
- Duplicate case labels are not allowed.
- Statements written before all the cases are ignored by the compiler.
- The break statement is optional.
- Case label cannot be a variable.

2.2 Iterative statements (Loop)

An iterative statement, or loop, repeatedly executes a group of statements, known as body of loop, until the controlling expression is false.

2.2.1 For Loop

- The for statement evaluates 3 expressions and executes the loop body until second controlling expression executes to false.
- It is recommended to use for loop when the number of iterations is known in advance.
- Syntax of for loop is:

```
for (expression – 1(optional); expression – 2(optional); expression – 3(optional))
{
    statement – 1
    statement – 2
    ⋮
    statement – n
    }
```

- for loop executes the loop body 0 or more times.
- for statements works as follows:

(a) expression – 1 is evaluated once before the first iteration of the loop.

(b) expression – 2 is a expression that determines whether to terminate the loop. expression – 2 is evaluated before every iteration.
If the expression is true (non - zero), the loop body executed.
If the expression is false (zero), execution of the for statement is terminated.

(c) expression – 3 is evaluated after each iteration.

(d) The for statement executes until expression-2 is false (0), or until a jump statement terminates execution of the loop.

- All 3 expression can be omitted

(a) If we omit expression-2, the condition is considered as true for an example:

```
for (i = 0;  ; i++)
statement.
```
represent an infinite loop

2.3 While Statement

- The while statement evaluates a controlling expression before every execution of the loop body.
- If the controlling expression is true (non-zero), the loop body is executed. If the controlling expression is false (zero), then the while statement terminates.
- Syntax:

```
while (expression)
statement

OR

while (expression)
{
statement-1
statement-2
⋮
statement-n
}
```

2.4 Do-while Loops

- Both for and while loop checks the loop termination condition before every iteration but do while loop check the condition after executing the loop body.
- do while loop body executes at least 1 time, no matter whether the loop termination condition is false or true.
- Syntax is:

```
do
statement
while (expression);

OR

do {
statement-1
statement-2
⋮
statement-n
} while (expression);
```

2.5 Break Statement

- The break statement provides an early exit from for, while and do while.
- A break statement causes the innermost loop or switch to be exited immediately.
- In implementation, when we know the maximum number of repetition but some condition is there, where we require to terminate the repetition process, then use break statement.

Example:

```
void main ()
{
int i = 1;
while (i < = 15)
    {
```

```
        printf("%d\t",i);
        if (i > 4)
            break;
        i = i + 1;
        }
    printf("End");
    }
    Output: 1 2 3 4 End
```

In above code, when i becomes 5, the condition i > 4 becomes true, so break statement executes & control passed outside of the loop body i.e., printf("End") executed.

2.6 Continue Statement

- Continue statement is related to break but it is not used that much frequently.
- Whenever continue is encountered in a loop, it skips the remaining statement of the current iteration and continues with the next iteration of the loop.
- Only applicable with loops, not with switch statement.

Example:

```
    void main () {
        int i = 1;
        for (i = 1; i < = 10; i++)
        {
            if(i%2 = = 0)
            continue;
            printf("%d",i);
        }
    }
    Output: 13579 (All odd numbers between 1 to 10)
```

whenever i becomes even, condition i%2 = = 0 becomes true & continue causes the control to go to i++ & printf is skipped.

❐❐❐

3 STORAGE CLASS & FUNCTION

3.1 Memory Organization of C Program

3.1.1 Code Segment

- It is also known as text segment.
- It contains executable instructions and this segment is a read only segment.
- Usually, this section is sharable.

3.1.2 Uninitialized data segment

- This section contains all static and global variables that are not initialized by the programmer and hence initialized to zero (default value).

3.1.3 Initialized data segment

- This section contains all static and global variables that are initialized by the programmer.

3.1.4 Stack

- In this local variables are stored and some other information is also saved.
- A set of values stored for a function is called as stack frame which atleast contain return address.

3.1.5 Heap

- This area is responsible for dynamic memory allocation. The heap area is managed by malloc(), calloc(), ralloc() and free() which may be use brk() and sbrk() system calls.

3.1.6 Static Memory

- Compile time allocation
- Static variable
- Global Variable

3.2 Storage Classes

Storage Class	Scope	Life Time	Default	Storage area
auto	Local	Within function	Garbage Value	RAM
static	Local	Till end of main program	0	RAM

extern	Global	Till end of main program	0	RAM
Register	Local	Within function	Garbage	Register

- Stack Overflow: Abnormal Termination
- If conflict between global and local variable occurs, then local variable gets preference.
- Declaration of a register variable is only a recommendation/request not a command.
- Cannot apply '&' operator with register variable.
- No physical memory is allocated using 'extern' keyword.
- Extern declaration is mandatory when an external variable is referred before it is defined or if it is defined in some other source file from the file where it is being used.
- Declaration of an external variable tells about the properties like its type, while definition leads to storage allocation.

3.3 Recursion

Definition: When the body of a function call the function itself directly or indirectly.

- Certain arguments for which the function does not call itself are called as base argument, base values.
- In general, for recursion to be non-cyclic whenever a function calls itself the formal arguments must get closer to the base argument.

Example 1:

Consider the factorial code:

```
int factorial (int n)
    {
    if (n = = 0)
    return 1;
    else
    return n * factorial (n - 1);
    }
```

If we call factorial (3), it will call factorial of 2 and so on …the argument will get closer to 0 i.e., the base argument.

- Recursion code is shorter than iterative code.
- Overhead is present.
- Some standard problems are best suited to be solved by using recursion for example- Tower of Hanoi, Factorial, merge Sort.

3.4 Static and dynamic scoping

3.4.1 Static Scoping

Static scoping is also called as lexical scoping. In this scoping, the binding of a variable can be determined by the program text. In this type of scoping compiler first looks in the current block (local), then in global variables i.e., local and then ancestors' strategy is followed.

Example 1:

```
int a = 10;
void main ()
{
    int a = 1;
    {
        int a = 2
        printf("%d",a);
        }
    }
```

Output: 2

As printf is referring to a, compiler first check in the current block & gets a variable whose value is 2.

Example 2:

```
int a = 10;
void main ()
{
int a = 1;
{
int b;
printf("%d",a);
}
}
```

Scope S_1

Scope S_2

Output: 1

Compiler looks for 'a' in scope S_1, because S_1 does not have variable a, it will look into higher scope S_2 that contains a variable name a whose value is 1.

3.4.2 Dynamic Scoping

In this type of scoping, the compiler first searches the current block and then successively searches all calling functions.

Example:

```
int i;
program main ()
{
    i = 10;
    call f ();
}
procedure f ()
{
    int c = 20;
    call g();
}
Procedure g() {
print i;
    }
```

Assuming that the above program is written in a hypothetical programming language which allows global variables

and dynamic scoping, let's try to find the output.
The order of function calls is:

$$\text{main ()} \xrightarrow{\text{calling}} \text{f()} \xrightarrow{\text{calling}} \text{g()}$$

g() is printing i, which is not present inside current block. So, because of dynamic scoping compiler will go to the function that calls g() i.e., compiler will search f() for variable i. Hence 20 is printed.

❐❐❐

4 ARRAYS AND POINTERS

4.1 Array

- An array is defined as the collection of similar type of data items stored at contiguous memory locations.
- Arrays are the derived data type in C programming language which can store the primitive type of data such as int, char, double etc.
- It has the capability to store the collection of derived data types such as pointer, structure etc.
- Each element of an array is of same data type and carries the same size example int = 4 byte.
- Elements of the array are stored at contiguous memory locations, meaning, the first element is stored at the smallest memory location.
- Elements of the array can be randomly accessed since we can calculate the address of each element of array with the given base address and size of the data element.

4.1.1 Advantages of C Array

1) Code Optimization: Less code to access the data.
2) Ease of traversing: By using the for loop, we can retrieve the elements of an array easily.
3) Ease of sorting: To sort the elements of the array, we need a few lines of code only.
4) Random Access: We can access any element randomly using the array.

4.1.2 Disadvantages of Array

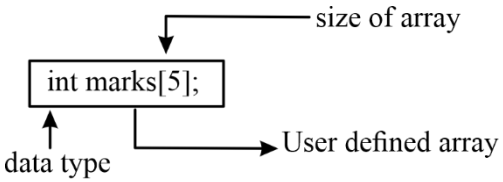
- **Fixed Size**: **1)** Whatever size, we define at the time of declaration of the array, we can't exceed the limit.
  2) It does not grow the size dynamically like linked list.

Declaration of C Array

Syntax:

```
data type array name [array_size]
```

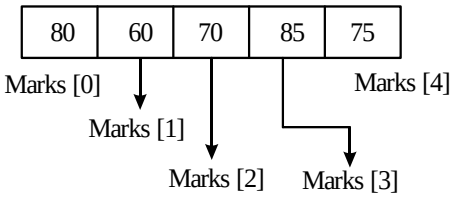
Example:



Initialization of C array:

We can initialize each element of the array by using the index. Suppose array int marks [5]. Then

```
marks [0] = 80
marks [1] = 60;
marks [2] = 70;
marks [3] = 85;
marks [4] = 75;
```



C Array Example:

```
# include <stdio.h>
int main ()
{
int i = 0;
int marks [5];
marks [0] = 80;
marks [1] = 60;
marks [2] = 70;
marks [3] = 85;
marks [4] = 75;
for(i = 0; i<5; i++)
{
    printf("%d\n",marks[i]);
}
return 0;
}
```

4.1.3 Array: Declaration with Initialization

- We can initialize the C array at the time of declaration.
  Example: int marks [5] = {20, 30, 40, 50, 60};
- In such case, there is no requirement to define the size.
  Example: int marks [ ] = {20, 30, 40, 50, 60}

Example:

```
# include<stdio.h>
int main ()
{
int i = 0;
int marks [5] =  {20, 30, 40, 50, 60}
for(i = 0; i < 5; i++)
    {
        printf("%d\n",marks[i]);
    }
return 0;
}
```

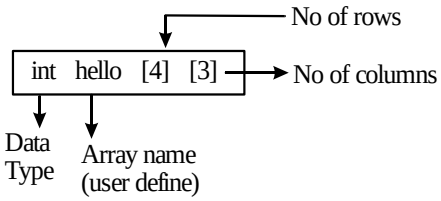
4.1.4 Two-Dimensional Array

- The two-dimensional array can be defined as an array of arrays.
- The 2D array is organized as matrices which can be represented as the collection of row and columns.

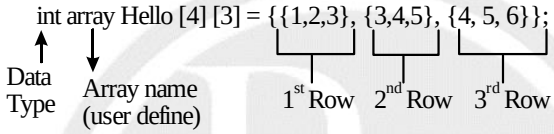
4.1.5 Declaration of two-dimensional array in C syntax

```
data_type array_name [rows] [columns]
```

Example:



Initialization of 2D Array



Example: Program of Two-Dimensional Array

```
# include<stdio.h>
int main()
{
    int i = 0, j = 0;
    int arrayHello[4][3] = {{1, 2, 3}, {2, 3, 4}, {4, 5, 6}};
    for(i = 0; i < 3; i++)
    {
    for(j = 0; j < 3; j++)
        {
            printf("arrayHello[%d][%d]=%d\n",i,j,arrayHello[i][j])
        }
    }
    return 0;
}
```

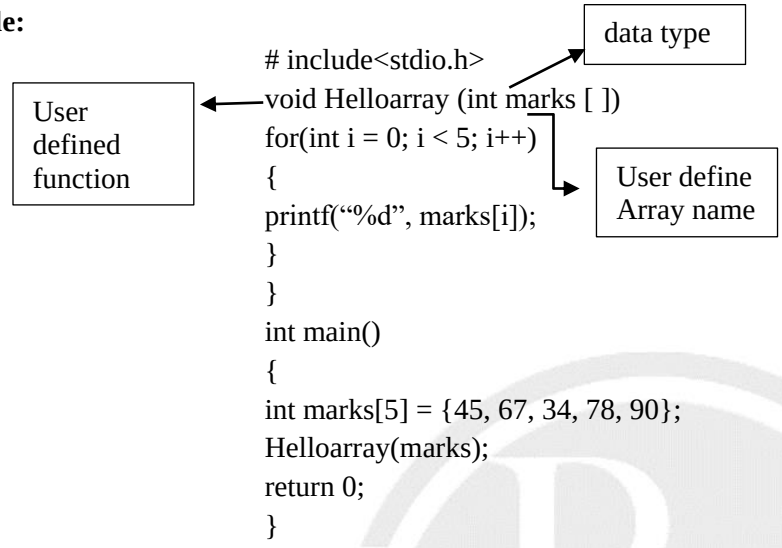
Output:

```
arrayHello[0][0] = 1
arrayHello[0][1] = 2
arrayHello[0][2] = 3
arrayHello[1][0] = 2
arrayHello[1][1] = 3
arrayHello[1][2] = 4
arrayHello[2][0] = 3
arrayHello[2][1] = 4
arrayHello[2][2] = 5
arrayHello[3][0] = 4
```

arrayHello[3][1] = 5
arrayHello[3][2] = 6

4.1.6 Passing array to a function

Example:




```
# include<stdio.h>
void Helloarray (int marks [ ])
for(int i = 0; i < 5; i++)
{
printf("%d", marks[i]);
}
}
int main()
{
int marks[5] = {45, 67, 34, 78, 90};
Helloarray(marks);
return 0;
}
```

Note: void Helloarray(int marks[ ])

Without return type function

4.1.7 Passing Array to a Functions as a pointer

Now, we will see how to pass an array to a function as a pointer.

```
# include<stdio.h>
void helloarray(char * marks)
{
    printf("Elements of array are:");
    for(int i = 0; i < 5; i++)
    {
    printf("%c", marks[i])
    }
    int main()
    {
    char marks[5] = {'A', 'B', 'C', 'D', 'E'};
    helloarray(marks);
    return 0;
    }
}
```

Note:
Whenever you pass an array, it is always call by reference. In previous 2 programs, we passed an address & formal argument in both the cases is a pointer internally.

How to return an Array from a function

```
# include<stdio.h>
int *fun()
{
int marks [5];
print f (“Enter the element in an array”);
for (int i = 1; i < 5; i++)
{
scanf(“%d”,& marks[i]);
}
return marks;
}
int main()
{
int *n;
n = fun();
printf(“\n Elements of array are:”);
for(int i = 0; i < 5; i++)
{
printf(“%d”,n[i]);
}
return 0;
}
```

Note:
fun() function returns a variable ‘marks’.
It returns a local variable, but it is an illegal memory location to be returned, which is allocated with a function in the stack. Since the program control comes back to the main () function, and all the variables in the stack are fraud.
Therefore, we can say that this program is returning memory location, which is already de-allocated, so the output of the program is a **segmentation fault.**

4.1.8 Array declaration and initialization

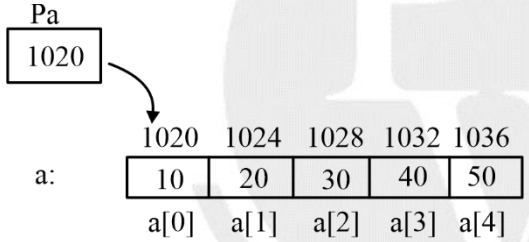
1. int A[ ] = {10, 20, 30}: valid
2. int A[3] = {10, 20, 30}: valid
3. int A[ ] ; invalid
4. int A[3] ; valid
5. int A[2][3]; valid
6. int A[ ] [3]; = {10, 20, 30}: invalid
7. int A[2] [3]; = {1, 2, 3, 4, 5, 6}: valid
8. int A[ ] [3]; = {1, 2, 3, 4, 5, 6}: valid
9. int A[2] [3] [2]; invalid
10. int A[ ] [3] [2]; invalid
11. int A[ ] [2] [ ]; invalid
12. int A[ ] [ ] [2]; invalid
13. int A[2] [3] [2]; = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12,}; valid

14. int A[ ] [3] [2]; = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12,}; valid
15. int A[2] [ ] [2]; = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12,}; invalid
16. int A[2] [3] [ ]; = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12,}; invalid

Note:
- while declaring an array, it is mandatory to provide the size of each dimension. (3), (6), (10), (11), (12) are not providing sizes of dimensions.
- While initializing an array, you have flexibility to omit only first dimension i.e., you are allowed to other dimension (1), (8),(14) are following this rule while (15), (16) are not following this rule.

4.1.9 Pointers

- In C, there is a strong relationship between arrays and pointers. An operation that can be achieved by arrays subscripting also be done with pointes.
- Consider the declaration
  int a[5] = {10, 20, 30, 40, 50};
  int* Pa;
  Pa = &a[0];
- Pointers are special variable that can hold address of other variables
  Syntax: data type * Identifier
  int *Pa: Pa is a pointer to integer variable
  i.e., Pa can hold address of integer variable



Pa contains address of a[0]

- Two operators are important to understand for understanding pointers: &, *
  i. &: address of operator
  ii. *: value at operator
- Pa is equivalent address 1020
- Pa is equivalent to value at (Memory location 1020)
  i,e *Pa is same as 10
- Pa is pointing to same element of array, then by definition Pa +1 points to next element, Pa + i points to elements after i elements before Pa.
  i.e, In our example Pa points to a[0]
  So, Pa+1 will point [1]
  Pa+2 will point to a[2]
  i,e
  Pa+i is address of a[i]
  $\Rightarrow$ Pa + i ≡ & a[i]
  $\Rightarrow$ *(Pa+i) ≡ *& a[i] ≡ a [i]
- Array name always represent address of the very first element
  i.e., Pa = &a[0];
  and

Pa = a ;

both are same.

- a[i] ≡ *(a+i) ≡ *(i + a) ≡ i [a]
  All are same and can be used interchangeably in program except in declaration.
- In declaration, we can not write
  int 3[a]; instead of int a[3]
- Array name represent a constant address.
  int a[10];

i. a++, ++a, – –a , a – – are all invalid as we cannot apply increment/decrement operator on constants:

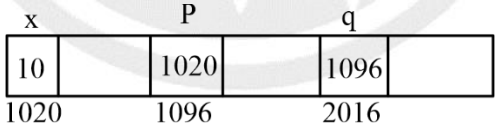
ii. Array name cannot be Lvalue of on assignment statement.
i.e.
array_name = any expression/address/value is invalid

- On the other hand, pointer being a variable, the following operations are valid.

```
int a[5]
int = *Pa;
Pa = a : valid
Pa+ + ;
Pa – –:
+ + Pa :
– – Pa :
```
} *all are valid*

- Multi-level pointer

```
int x      :   x can store value
int *P     :   p can store address of integer variable
int **q    :   q can store address of pointer variable
x = 10     :   valid
p = &x     :   valid
q = &p     :   valid
```



p ≡ 1020 i, e address of x
*P ≡ value at (memory location 1020)
*p ≡ 10
q ≡ 1096 i,e address of variable p
*q ≡ value at (memory location 1096)
≡ 1020 which is again an address
*q ≡ value at (memory address 1020)
**q ≡ 10

1. **Dangling pointer:** The pointer pointing to a deallocated memory block is known as dangling pointer.
   This situation raises an error known as dangling pointer problem. Dangling pointer occurs when a pointer pointing to a variable goes out of scope or when a variables memory gets deallocated.
   Consider the code.
   int *f( ) {

```
int a = 10; // a is a local variable and goes out of scope after execution of f( )
return &a :
}
int main () {
int * ptr = f (); //Ptr points to something which is not valid
print ("/d"*,ptr);
return 0;
}
```



- To overcome this problem, just make variable a as static when x become static it has scope throughout the program.

2. **Uninitialized pointer:** An uninitialized pointer also known as wild pointer
   A pointer which has not been initialized to anything can be dangerous

- The value saved in an uninitialized pointer could be randomly pointing anywhere in memory
- int *p



- int *p



- Storing a value using an uninitialized pointer has the potential to overwrite anything in your program including your program itself.
- System may crash.
- Always initialize with NULL.

3. **NULL pointer:** It is a pointer which is pointing to nothing It is a specially designed pointer that stores a defined value but not a valid address of any element or object.
   NULL pointer does not hold valid address and that is why if you try to dereference it, you will get an error

```
int *p = NULL:
printf ("%d",*p) :        //NULL pointer dereferencing
```

4. **void pointer**: void pointer is a pointer that points to some location in memory but is does not have any specific type.
   It can point to any type of data and any pointer type is convertible to a void pointer.
   Its declaration:

```
void *ptr :
```

   is saying that ptr is a pointer that can hold an address
   cannot be dereferenced directly
   i.e.,

```
void *ptr
int x = 10:
```

```
ptr = &x :
printf ("%d", *ptr);        //invalid
```

you need to typecast the void pointer before dereferencing.

i.e.,

```
void *ptr
int x = 10;
ptr = &x;
printf("%d",*(int*)ptr);
                 └→typecasting
```

Here, (int*)ptr does type costing of void

(int) ptr dereferences the typecasted void pointer variable

- Pointer arithmetic in not possible on void pointer.
  i,e……. ptr+ +, + + ptr, – –ptr, ptr – –, ptr +1, ptr -1 is not allowed an void pointer.

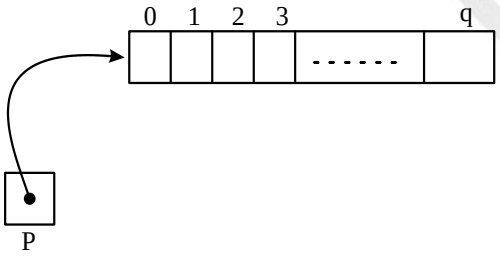
- An uninitialized pointer holds a garbage value while a NULL pointer holds a defined value but not a valid address.

4.1.10 Memory leakage problem

- Whenever a pragrammer allocated memory dynamically then it is the reponsibility of the programmer to free that memory after usage.
- Memory leakage problem occurs when a programmer allocates memory dynamically but does not de-allocate it after using it.
- It reduces the performance of the computer by reducing the amount of availbale memory.

4.1.11 Understanding Declartion

1. int *(P[10]) : P is an array of 10 pointer to integer
2. int (*P)[10] : P is a pointer to an array of 10 integers



3. int (*P)() : P is pointer to a function that takes no argument and returns an integer.
4. int (*P) (int, int) : P is a pointer to a function that takes 2 integer arguments and returns an integer.
5. int *P (char*) : P is a pointer to a function that takes a pointer to character argument and return a pointer to integer.

4.1.12 Pointer to Functions / Function Pointer

- Just like normal pointers we can have pointers to functions.

```
int(*p)(int, int)
```

P is a pointer to a function that take 2 integer arguments and returns an integer value.

- **Declaration of function pointer:** declaring a pointer to function is almost same as declaring the fuction except that in function pointer are prefix is with an asterik * symbol.
  For example: if the function is

  int add(int, int)

  Declaration of a fuction pointer for add() fuction is :

  int (*ptr)(int, int);
- How to call a function through function pointer : In order to all a function using function pointer, first we need to assign the address of function code to the pointer

Ex 1:

```
int add (int, int):
void main () {
        int (*ptr) (int, int);
        ptr = &add;       // ptr is a pointer to add() function
printf ("The sum of 10 and 20 is", (*ptr )(10, 20)); //calling add()
}
int add (int, int);
```

Ex 2:

```
void main() {
        int (*ptr) (int, int);
        ptr =  add ; // same as ptr = & add
printf (" The  sum of  10 and 20 is", (*ptr) (10, 20));
```

- A function can be called using following 4 ways using pointer to fuction.

int add (int int);
void main() {
int (*ptr) (int, int);
ptr = &add;
printf ("%d", (*ptr)(10, 20));
}
	int add (int int);
	void main() {
	int (*ptr) (int, int):
	ptr = add;
	printf ("%d", (*ptr) (10, 20));
	}

int add (int, int) ;
void main() {
int(*ptr) (int, int);
prt= &add;
printf ("%d", (*ptr)(10, 20));
}
	int add (int, int);
	void main() {
	int (*ptr) (int, int);
	prt= add;
	printf ("%d", (*ptr) (10, 20));
	}

- Using function poitnter we are able to pass a function as an argurment to other function and can also be returned from function.
  Array of function pointer
  int (*ptr[3])(int, int)
  Ptr is an array of 3 pointers to a function that takes 2 arguments and returns an integer.

❐❐❐

5 STRINGS

5.1 Strings

- Sequence of characters terminated by a null character '\0'.
  Syntax:
  char str_name[size];
- Initializing a string in c
  - char str[] = "Pankaj";
    Here, str is internally a pointer (holds the address of first character)
  - char str[20] = "Pankaj";
    Here, we predefine the size as 20 but we should always take one extra space along with size of string.
    i.e atleast 7 size must be there to store "Pankaj" (6+1)
  - char str[7] = {'P', 'a', 'n', 'k', 'a', 'j', '\0'};
    must set the end character as '\0'
  - chat str[] = {'p', 'a', 'n', 'k', 'a', 'j', '\0'};

5.1.1 Reading a string

- **scanf():**
  when we use scanf() to read, we use %s as a format specifier without using "&" to access the variable address because an array name acts as a pointer.

```
#include <stdio.h>
    int main (){
        char name [20];
        printf("enter your name \n");
        scanf("%s",name);
        printf("Hello %s",name);
          }
```

Output:

```
Enter your name
Pankaj Sharma
Hello Pankaj      // why not Hello Pankaj Sharma
```

The above code did not print Hello Pankaj Sharma as expected because scanf halts reading as soon as a whitespace or a newline character is encountered, and that is why it reads only Pankaj.

- In order to read a string containing space, we use gets() function. gets() ignores the whitespace & stops reading when a newline is found.

```
# include<stdio.h>
```

```
void main(){
        char name [20];
        printf("Enter your name :");
        gets(name);
        printf("Hello %s", name);
        }
Output :
        Enter your name: Pankaj Sharma
        Hello Pankaj Sharma
```

- A string literal always represent base address of a string. i.e address of first character.
- "Pankaj" represents address of character 'P'

Hence "Pankaj" [0] means 'P'

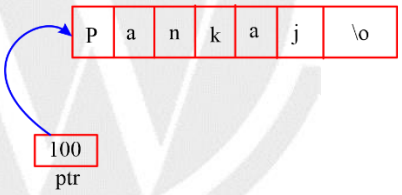
"Pankaj" [1] ≡ *("Pankaj"+1) ≡ *(Address of 'P'+1)
≡ *(Address of 'a')
≡ 'a'

5.2 Strings and pointers

- Array name represents the address of its first element. Similar to character arrays, we can create a character pointer to represent a string that will hold the starting address i.e address of first character of string.

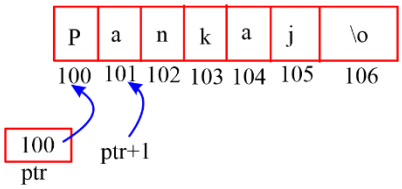
Example - 1

```
# include<stdio.h>
void main(){
        char *ptr = "pankaj"
        printf("%s",ptr);
        }
Output : pankaj
```



Example – 2

```
# include<stdio.h>
void main(){
        char *ptr = "Pankaj"
        printf("%s",ptr+1);
        }
Output: ankaj
```



5.3 Predefined functions for string

(1) strlen (): returns the length of the string passed as an argument.

(2) strcpy (): copies the contents of one string to another. strcpy (S_1,S_2) copies the content of string S_2 to string S_1.

(3) strcmp (): compares the first string with the second string. If both are same it returns 0.
strcmp (str1,str2) returns 0 if both strings are same.

(4) strcat (S_1,S_2): concat S_1 string with S_2 string and the result i.e concatenation of S_1,S_2 is stored in S_1.

5.4 Important concepts

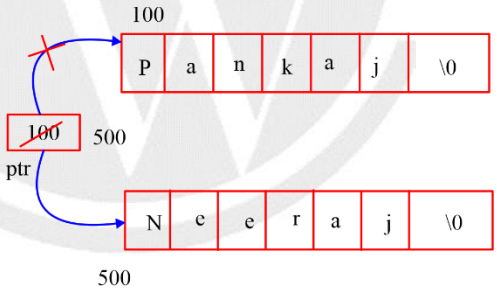
(1)
```
# include<stdio.h>
void main(){
        char name [20] = "Pankaj" ;
        name = "Neeraj" ;              // Error
        printf("%s", name);
                }
```

- Array name being a constant can't be presented at the left side of the assignment statement i.e it can't be L-value. You can't re-assign another string to array.

(2)
```
# include<stdio.h>
void main(){
                char name [20] = "Pankaj" ;
                name [1] = 'u' ;
                printf("%s", name);
                }
```
Output: Punkaj

- We are allowed to change content of array and hence we can update any character.

(3)
```
# include<stdio.h>
void main(){
                char * ptr = "Pankaj" ;
                ptr = "Neeraj" ;
                printf("%s", ptr);
                }
```
Output: Neeraj

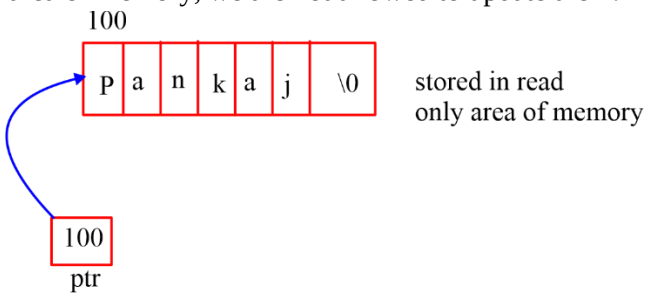


- Pointer ptr being a variable can hold different address at different time. Hence, we can assign another string to a character pointer any time.

(4)
```
# include<stdio.h>
void main()
        {
        char * ptr = "pankaj" ;
        ptr [1] = 'u' ;  // Invalid
        printf("%s", ptr);
        }
```

- String literals are stored in read only memory area i.e "pankaj" is stored first & then the address of character 'p' is given to ptr.

- As they are stored in read only area of memory, we are not allowed to update them.



❐❐❐

6 TYPES OF DATA STRUCTURE, ARRAY & LINKED LIST

6.1 Introduction

6.1.1 Linear data structure

Every element can have almost 2 neighbours. Ex. arrays, linked list, stack, queue.

6.1.2 Non-Linear data structure

Element can have more than 2 neighbours. Ex. Tree, graph.

6.2 Arrays

6.2.1 1-D array

Theoretically index can start from any integer value. Let A be a 1-D array of n elements and the size of each element is w bytes, then the address of A[i] element.

$$\text{Base Address}\,(A) + (i - \text{starting index}) * w$$

6.2.2 2-D Array

Analogous to matrix with rows and columns. Can be implemented in RMO (Row-major order) or CMO (Column-major order).

Let A[M] [N] be a 2-D array, where size of each element is w-bytes, then the address of A[i] [j] is :

(a) In RMO

Address (A[i] [j]) = Base Address (A) + [(j-starting index) + (i – starting index) × M]xw

(b) In CMO

Base Address (A) + [(i – starting index) + (j – starting index) × M] × w

6.2.3 Sparse Matrices

Most of the element of the matrix have 0 value and representing such matrix by a 2D array leads to wastage of memory and that is why, it is better than we will only store non-zero elements (Efficient method).

6.2.4 Lower Triangular Matrix

$n \times n$

$A_{ij} = 0 \quad \text{if} \quad i < j$

$A_{ij} \neq 0 \quad \text{if} \quad i > j$

For ex.

$$A=\begin{bmatrix} A_{11} & 0 & 0 & 0 \\ A_{21} & A_{22} & 0 & 0 \\ A_{31} & A_{32} & A_{33} & 0 \\ A_{41} & A_{42} & A_{43} & A_{44} \end{bmatrix}$$

6.2.5 Upper triangular Matrix

$$A=\begin{bmatrix} A_{11} & A_{12} & A_{13} & A_{14} \\ 0 & A_{22} & A_{23} & A_{24} \\ 0 & 0 & A_{33} & A_{34} \\ 0 & 0 & 0 & A_{44} \end{bmatrix}$$

6.2.6 Tri-diagonal Matrix

Matrix that has non-zero elements only in the main diagonal, diagonal just below main diagonal and diagonal just above main diagonal. All other elements are zero.

$$\begin{bmatrix} A_{11} & A_{12} & 0 & 0 & 0 \\ A_{21} & A_{22} & A_{23} & 0 & 0 \\ 0 & A_{32} & A_{33} & A_{34} & 0 \\ 0 & 0 & A_{43} & A_{44} & A_{45} \\ 0 & 0 & 0 & A_{54} & A_{55} \end{bmatrix}$$

6.2.7 Efficient method to store sparse matrix

Using Row-major order for string lower triangular matrix, the element at index (i, j) can be represented as :

$$\text{Index of } A_{ij}=\left[\frac{i(i-1)}{2}+(j-1)\right]$$

Using column-major order for storing upper triangular matrix, the element at index (i, j) is sparse matrix can be represented as:

$$\text{Index of } A_{ij}=(i-j)+\left[(j-1)N-\frac{(j-i)(i-2)}{2}\right]$$

Matrix order

N × N

Using Row-major order for storing upper triangular matrix, the element at index (i, j) in sparse matrix can be represented as :

$$\text{Index of } A_{ij}=(j-i)+\left[(i-1)N-\frac{(i-i)(i-2)}{2}\right]$$

Where N : Number of rows/columns

Using column-major order for starting upper-triangular matrix the element at index (i, j) can be represented as

$$\text{Index of } A_{ij} = \left[(i-1) + \frac{j(j-1)}{2} \right]$$

Using Row-major order for storing tri-diagonal matrix,

Index of element A_{ij} = $2i + j - 3$

Using column-major order for storing tri-diagonal matrix,

Index of element A_{ij} = $i + 2j - 3$

6.3 Linked List

6.3.1 Linked List

Linked list is collection of elements called node, where each node contains atleast 2 fields.

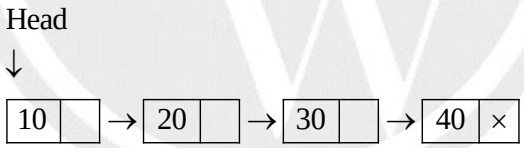
(i) Data field : that contains information.

(ii) Link field : contains address of next note.

6.4 Types of Linked List

6.4.1 Singly Linked List

Every node contains data and a pointer to the next node in the linked list, we can traverse the linked list only in forward direction.



- Head : Pointer that contains address of 1st node.
- Head contains NULL represent empty Linked List.
- If Ptr contains address of some note (Ptr is a pointer to node) then to go to the next node, we need.

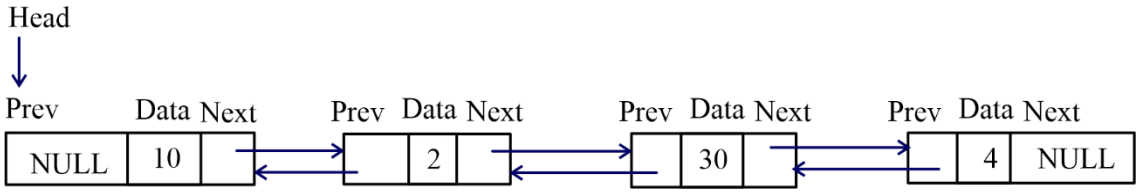
$$\text{Ptr} = \text{Ptr} \rightarrow \text{Link}$$

- Node can be implemented by structure in C

```
Struct Node {
        int data;
Struct Node    * Link;
        }
```

6.4.2 Doubly Linked List

A two-way linked list in which every node contains a pointer to the next node as well as pointer to previous node in the list we can traverse it in backward direction also.



Structure of a Node :

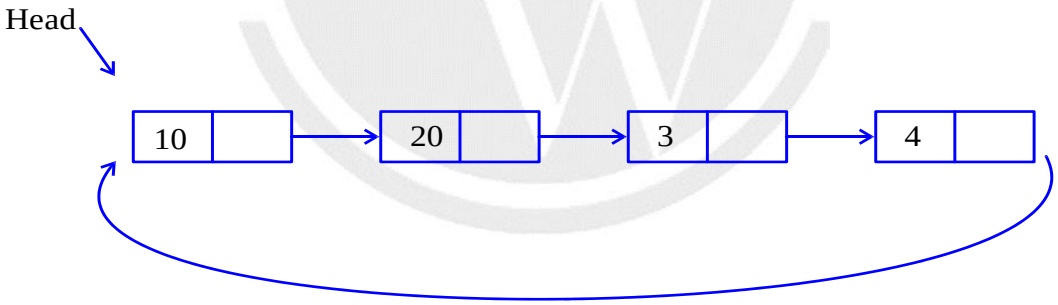
```
Struct Node {
        struct Node * Prev;
        int data;
        struct node * next;

        };
```

6.4.3 Circular Linked List

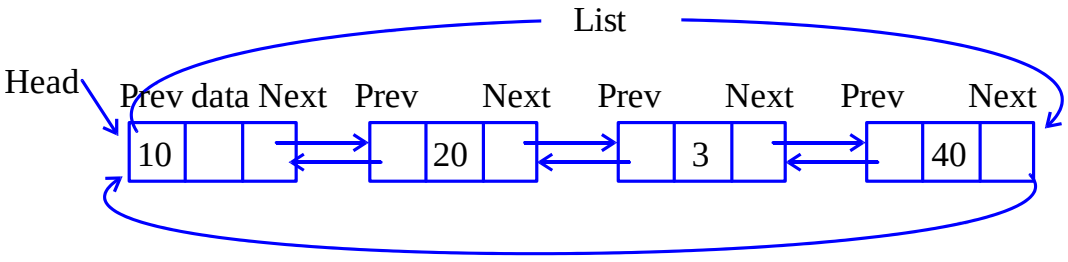
A linked list in which last node points back to the first node in the list.

- Circular linked list has no beginning and no end.



6.4.4 Doubly circular linked list

It is a 2-way circular linked list.

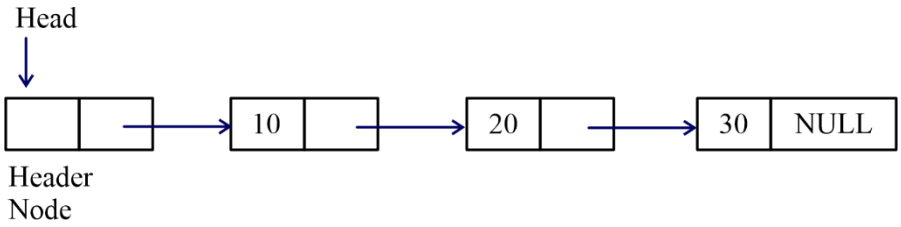


6.4.5 Header Linked List

A linked list that contains a special node called header node at the beginning of the list.

- Head will not point to first node.

- Head points to the header node.



6.4.6 Applications

- used to implement other data structure link stack, queue data structure.
- used for dynamic memory allocation.

6.4.7 Advantages and Disadvantage

- Insertion and deletion are efficient.
- Using pointers in every node, require more memory.
- Random accessing in not possible.
- Circular linked list can be used to implement Fibonacci Heap.

❑❑❑

7 STACK AND QUEUE

7.1 Introduction

Linear data structure in which both insertion and deletion operation are performed at one end called TOP of the stack. Works on LIFO (Last In First Out) Policy.

7.1.1 Application

To post-pone certain decision (To wait/To delay)

- **Stack Permutation :** Order of insertion of key is fixed but we can pop an element any time.

  Number of possible stack permutation with n elements $= \frac{^{2n}C_n}{n+1}$

- Infix to postfix
- Infix to prefix
- Prefix Evaluation
- Postfix Evaluation
- Recursion
- Tower of Hanoi

7.2 Queue Data Structure

Linear data structure in which insertion is done at one end called rear of the queue and deletion is performed at other end called front end of the queue.

Works on FIFO (First In First Out) policy.

7.2.1 Applications of Queue Data Structure

- Whenever a resource is being shared among many users.
- When data is transferred asynchronously.
- FCFS scheduling
- Algorithms like BFS.

7.3 Implementation

Can be implemented using arrays, linked list.

(a) Using Array :

- Easy to implement and memory efficient.
- Not dyanmic

(b) Using Linked List :

- Dynamic
- Require extra memory because of pointer field.

7.4 Standard Operations

7.4.1 Stack

(a) Push ( ) : Insertion of an element onto stack.

(b) Pop ( ) : Deletion of an element.

(c) Is_empty ( ) : Returns true if stack is empty otherwise return false.

7.4.2 Queue

(a) Enqueue : Insertion of an element.

Performed at rear end of the queue.

(b) Dequeue : Deletion of an element.

Performed at front end of the queue.

7.5 Circular Queue

Extended version of simple queue in which last element is connected to the first element.

In simple queue, even though space is available, we are not able to insert a new element and declaring it a overflow. Circular queue solves this problem.

Queue Full

(i) Front = = 0 & & Rear = = SIZE – 1 OR

(ii) Front = = (Rear + 1)% SIZE

7.6 Priority Queue

A queue in which a priority is associated with every element and elements are processed according to their priorities and if two elements have same priority then they are processed as per their arrival in the queue.

❑❑❑

8 TREE DATA STRUCTURE

8.1 Introduction

8.1.1 Terminology

(i) Internal Node : Node with atleast 1 child.

(ii) Leaf Node : Node without any child.

(iii) Root Node : Only node that does not have any parent.

(iv) Complete binary Tree : Binary tree in which nodes are inserted from left to right at every level and we can not insert at node at (K+ 1)th level until all levels upto K are fully filled.

(v) Full Binary Tree : A binary tree in which every internal node has exactly two children and all the leaf nodes are at some level.

Binary tree that contains maximum number of nodes.

(vi) Strict Binary Tree : A binary tree in which every internal node has exactly two children.

(vii) Complete K-ary tree : Tree in which every internal node has exactly K-childs.

8.1.2 Mathematical Result

(i) For a binary tree of height h

- Maximum number of nodes = $2^{(h+1)} - 1$
- Minimum number of nodes = h + 1

(ii) For a complete binary tree of height h

- Maximum number of nodes = $2^{(h+1)} - 1$
- Minimum number of nodes = 2^h

(iii) For a full binary tree :

Number of nodes = $2^{h+1} - 1$

(iv) Total number of unlabelled binary trees with n nodes = $\frac{{}^{2n}C_n}{n+1}$

(v) Total number of labelled binary tree with no keys = $\frac{{}^{2n}C_n}{n+1} \times n!$

(vi) Total number of binary trees with a given preorder (n length) = $\frac{^{2n}C_n}{n+1}$

(vii) Total number of binary trees with a given proorder/postorder/inorder = $\frac{^{2n}C_n}{n+1}$

(viii) Number of binary trees with a given preorder and inorder = 1

(ix) Number of binary trees with a given postorder and inorder = 1

(x) Number of leaf nodes = Number of nodes with two children + 1

8.2 Binary Tree Traversal

(1) Preorder :

(i) Process the root.

(ii) Traverse the left subtree of root in preorder.

(iii) Traverse the right subtree of root in preorder.

(2) Inorder

(i) Traverse the left subtree of root in inorder.

(ii) Process the root.

(iii) Traverse the right subtree of root in inorder.

(3) Post-order

(i) Traverse the left subtree of root in post-order.

(ii) Traverse the right subtree of root in post-order.

(iii) Process the root.

In-order

- **Morrison Traversal :**

  In-place traversal of a binary tree

  No recursion

  Constant extra space.

8.3 Binary Search Tree

A binary tree in which every node satisfy the property that all the keys in the left subtree of the node are smaller than node's key and all the keys in the right subtree of the node are greater than node's key.

- Inorder traversal of a binary search tree is ascending order of keys in the BST.
- Insertion in a BST
  (a) O (log n) best case
  (b) O (n) worst case.

- Deletion from a BST :

  (a) Deletion of leaf node : can be done directly

  (b) Deletion of node with one child : copy the child to the node and delete the child.

  (c) Deletion of a node with two children : first find the inorder successor of the node, copy the contents of this successor to the node and dlete the inorder successor.

  We can also use inorder predecessor.

  - TC : O (log n) best case

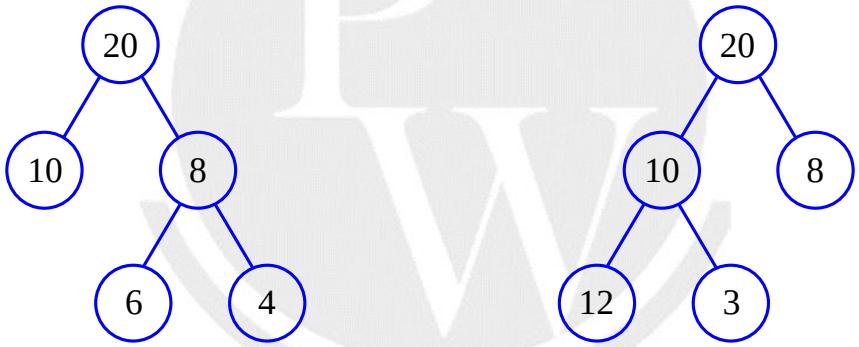
    O (n) worst case

8.4 Heap

A complete binary tree in which every node satisfy heap property (min heap or max heap)

(i) Min heap : A complete binary tree in which every node satisfy the property that the value of the node is smaller than both its children.

(ii) Max heap : A complete binary tree in which every node satisfy the property that the value in the node is greater than both its children.



Not a heap Not a heap

- **Construction of Heap :**

  (i) By inserting keys one after another : O (n log n)

  (ii) Using build-heap method/heapify algo : O(n)

- Number of min-heaps possible with n keys

  $T(n) = T(k) \cdot T(n - k - 1) \cdot {}^{n-1}C_k$

  K : Number of elements in the left subtree of root node.

- Deletion :

  (1) Swap A[1], A[n]

  (2) Apply Heapify on A[1] considering there are only (n – 1) nodes.

  T.C = O (log n)

- Heapsort (Assuming max heap)

  (i) Build Max Heap O(n)

  (ii) Replace root element with last element $\left(A[1] \leftrightarrow A[n]\right)$ reduce the heap size by 1 and then apply heapify at root node.

  (iii) Repeat step 2 while heap size is greater than 1.

8.5 AVL Search Tree

- Height balanced BST.
- Balance factor (B_f) is given by

$$B_f = |h_L - h_R| \leq 1$$

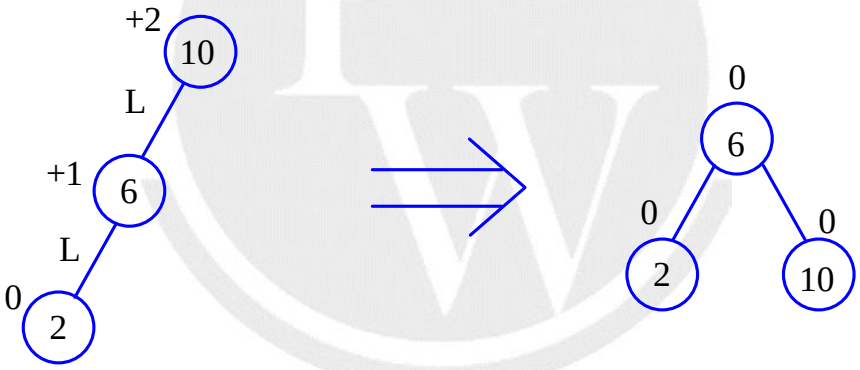
i.e., Balancing factor of a node can be +1, –1 or 0.

8.5.1 Insertion

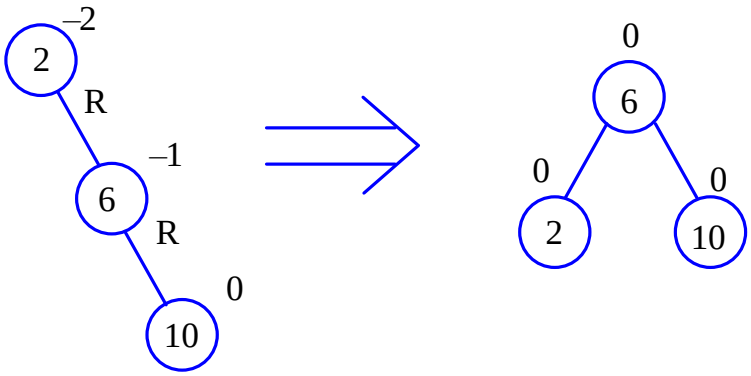
To ensure that the tree remains AVL tree, after insertion some re-balancing is performed i.e., certain rotations are needed.

8.5.2 Types of Rotation

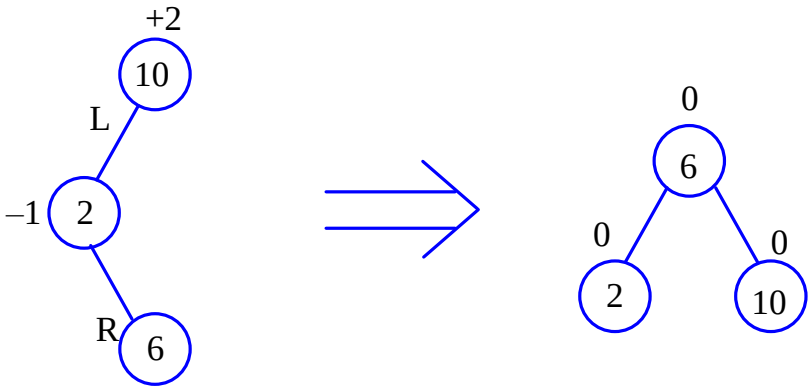
(i) Left-Left (LL) Rotation :



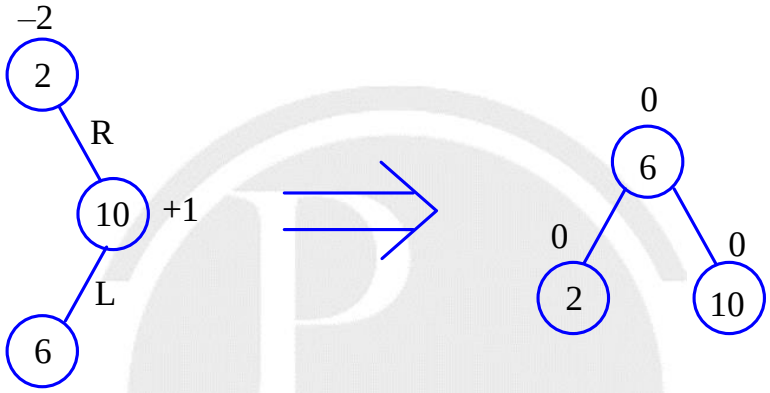
(ii) Right-Right (RR) Rotation :



(iii) Left-Right (LR) Rotation :



(iv) Right-Left (RL) Rotation :



- LL, RR are single rotation.
- LR, RL are double rotation.
- Just work no tri-node structure.
- Minimum number of nodes in an AVL tree of height h is given by recurrence.

  $n(h) = 1 + n(h - 1) + n(h - 2)$ $\quad h \geq 2$

  $n(0) = 1$

  $n(1) = 2$

  i.e., it also forms a Fibonacci series

  1, 2, 4, 7, 12, …….

❑❑❑

09 GRAPH TRAVERSAL

9.1 Introduction

9.1.1 Breadth First Search

- Uses a queue data structure.
- To find whether a graph is connected or not.
- To find number of connected components in a given graph (Breadth First traversal is used)
- To detect cycle in a graph.
- To find whether a given graph is bipartite or not.
- T.C. = O (V + E) using Adjacency List

  = $O(V^2)$ using Adjacency matrix.
- To find shortest path in undirected graph without weights.

9.1.2 Depth First Search

- Uses stack.
- To detect a cycle in a graph.
- To find whether a graph is connected or not.
- To find whether given graph is bipartite or not.
- To find number of connected components in the graph.
- To find bridges in the graph.
- To find articulation points in the graph.
- To find topological sort of a graph.
- To find biconnected components.
- To find strongly connected components.

❑❑❑

10 HASHING

10.1 Introduction

- Searching technique.
- Mapping keys into hash table using a hash function.
- Efficiency based on hash function used for mapping.

10.1.1 Terminology

(i) Collision : When two keys mapped to same location, collision occurs.

$H(k_1) = H(k_2)$, where H(k) is the hash function used.

(ii) Load factor $\lambda = \dfrac{\text{no. of keys}}{\text{Hash table size}}$

10.1.2 Collision resolution technique

(i) Open addressing (closed hashing)

(a) Linear probing
(b) Quadratic probing
(c) Double hashing

$\Big\} \lambda \le 1$

(ii) Separate chaining

10.1.3 Linear Probing

Whenever there is a collision at memory location 'L', then we will search for empty slot sequentially starting from 'L' then L + 1, L + 2, L + 3, …….

10.1.4 Problem with linear Probing

Linear probing suffers with primary clustering problem, consecutive keys forms a cluster and the time to find a free slot increases.

10.1.5 Quatratic Probing

Hash function $h(x) = x$ modm. It leads to a collision and it is i^{th} collision for key x then the collision resolution function is given as:

$H(x, i) = (h(x) + i^2)$ modm

Searching order : L, L + 1, L + 4, L + 9, ……

- Free from primary clustering
- Suffers with secondary clustering problem.

10.1.6 Double Hashing : Using 2 hash functions

(hash – 1 (key) + *i* * hash-2 (key))% m

M : table size

Hash-2 (key) : Secondary hash function must not give output 0.

* One of the best probing.
* Uniform distribution.
* Tree from primary and secondary clustering.
* Computation overhead because of two hash function computation.

10.2 Chaining

Uses the concept of linked list (chain) when more than one element are hashed to same location (slot) then elements are inserted into a singly linked list (chain).

- Deletion is easy compared to open addressing.
- Deleting in open addressing require rehashing remaining keys.

❑❑❑

GATE 2027 CS — Chapter 1: C Programming, from zero to PYQ-ready

A self-contained chapter: no book, no coaching needed. Story → core → GATE pattern → solved questions → traps → quiz.

You said your BTech basics faded and you can't sit with a thick book — so this chapter works differently. Every concept is told as a **story** first, then compressed to the **exact bookish truth** GATE wants, then drilled with **solved exam-pattern questions**. GATE directly asks 4–6 marks of pure C every year, and C is the language of almost every DSA/code question — so this chapter is your foundation.

1.1 Anatomy of a C program — who does what before your code runs

Real-life story: you write a letter in Hindi for a foreign friend. A **helper** pastes the standard greeting at the top (preprocessor + #include), a **lawyer** checks every sentence obeys grammar (compiler), a **typesetter** converts it to print symbols (assembler), and the **post office** collects everything with addresses into one parcel (linker) that finally gets delivered (the .exe runs).

■ THE BOOKISH CORE (what the textbook says)

- Source file **.c** → **Preprocessor** (handles #include, #define — pure text copy-paste) → **Compiler** (checks syntax, translates to assembly **.s**) → **Assembler** (machine code **.o**) → **Linker** (joins .o files + libraries like printf) → executable.
- **#include <stdio.h>** literally pastes stdio.h's declarations at the top of your file.
- Errors: syntax errors caught at compile time; linker errors when a function is declared but its code is missing; runtime errors while executing (e.g., divide by zero — C doesn't even stop you!).

■ WHAT GATE ASKS (patterns from 2007–2026 papers)

- Which stage expands #define? → **Preprocessor** (GATE asks this almost every other year).
- A .o file contains? → **object / machine code**, not yet an executable — linking is still pending.

```
#include <stdio.h> // helper pastes declarations of printf
int main(void) { // every C program starts here
printf("GATE 2027!\n");
return 0; // 0 = success report to the OS
}
```

Output: **GATE 2027!** (\n is the newline escape. Note: main returns int — the old void main() habit is wrong for GATE.)

1.2 Variables, types & constants — lockers, values and sizes

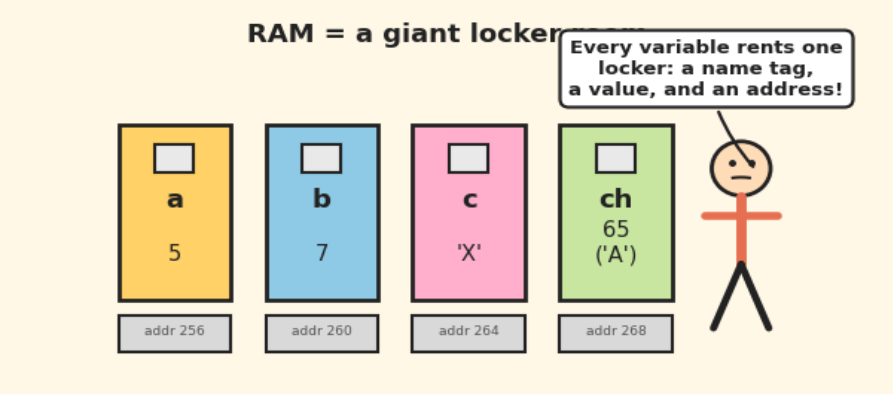


Fig 1.1 — Every variable = one locker: name tag, value, address.

■ **THE BOOKISH CORE (what the textbook says)**

- Typical sizes on 64-bit Linux/GCC (the GATE reference platform): **char 1, short 2, int 4, long 8, float 4, double 8, pointers 8** bytes.
- **sizeof(x)** gives size in bytes, type size_t — print with %zu. It is computed at compile time (no runtime cost), EXCEPT for variable-length arrays.
- A character constant **'A'** is stored as its ASCII code (**'A'=65, 'a'=97, '0'=48**). And in C, **'A' has type int** — that's the trick behind the most repeated C question in GATE history (see below).
- Integer division: **5/2 = 2** (fraction dropped), 5.0/2 = 2.5. C99 rule for negatives: division truncates toward zero: **-7/2 = -3, -7%2 = -1**.
- Unsigned arithmetic wraps around by definition: **unsigned int x = 0; x--;** makes x = 4294967295. Signed overflow is undefined behaviour.

■ **WHAT GATE ASKS (patterns from 2007–2026 papers)**

- **The sizeof trio (asked 2008, 2011, and repeatedly):** what is printed?

```
printf("%zu %zu %zu", sizeof(char), sizeof('A'), sizeof("A"));
```

Answer: 1 4 2. sizeof('A') = 4 because a character *constant* is an int in C (only in C++ is it 1). sizeof("A") = 2 because a string literal is a char array **including the hidden '\0'**: 'A', '\0'. One line of code, three exam tricks.

■■ **TRAP ALERT (where rank-hopefuls bleed marks)**

- Writing **5/2** when you meant 2.5 — GATE loves hiding this inside a bigger expression.
- Assuming char is signed or unsigned — it is **compiler-dependent**.

1.3 Operators — the ++/-- drama, comma and short-circuit

Real life: a++ is a train ticket — you pay the old fare, and the conductor updates the fare AFTER your trip. **++a** is the opposite: the fare updates FIRST, then you pay. Both end at the new fare; only what you "used" differs.

■ **THE BOOKISH CORE (what the textbook says)**

- Priority (high → low): postfix ++/-- and function calls → unary (++, --, *, &, !, sizeof, cast) → * / % → + - → shifts → relational → equality → & → ^ → | → && → || → ?: → = → comma. Assignment and conditional group RIGHT-to-left; most binary operators group LEFT-to-right.
- **Comma operator** evaluates left to right, throws all results away except the last: **int x = (1, 2, 3);** → x = 3. The commas inside function arguments are NOT comma operators.
- **Short-circuit:** in **A && B**, if A is 0, B never runs (saves time and guards crashes: **p != NULL && *p == 5**). In **A || B**, if A is true, B never runs.

■ **WHAT GATE ASKS (patterns from 2007–2026 papers)**

- Defined-output questions only: "int i = 5; j = i++ + 10; → j = 15, i = 6." "k = ++i + 10; → k = 16, i = 7." Trace value AND variable separately.
- Short-circuit state questions: "int i = 0; int j = 0 && i++; printf('%d %d', i, j);" → **0 0** — i++ never executed, j = 0.

```
int a = 10;
int b = a++ + 5; // b uses OLD a: b = 15, then a becomes 11
int c = ++a + 5; // a becomes 12 first: c = 17
printf("%d %d %d", a, b, c);
```

Answer: 12 15 17. Read every ++/-- in two beats: "what value is USED here" and "what value does the variable end with".

■■ **TRAP ALERT (where rank-hopefuls bleed marks)**

- **a++ + ++a and i = i++ are UNDEFINED BEHAVIOUR** — you modify a twice between sequence points. GCC prints something, but "compiler dependent" / "undefined" is the GATE-correct answer. If two options are numbers and one is "undefined/compiler-dependent", strongly suspect the question is testing exactly this.
- Confusing the comma operator with separating arguments: f((a, b)) passes one value; f(a, b) passes two.

1.4 Control flow — the semicolon that ate 10 marks

■ **THE BOOKISH CORE (what the textbook says)**

- **if/else:** a "dangling else" always pairs with the **nearest unmatched if** — indentation lies, braces don't.
- **switch:** without break, execution falls through to the next case like dominoes. break stops the fall; default catches whatever remains.
- Loops: **for(init; cond; update)**. **for(;;)** = infinite loop. A semicolon right after for/while closes the loop body into an EMPTY statement — the loop runs on air, then control continues after it.

■ **WHAT GATE ASKS (patterns from 2007–2026 papers)**

- **A genuine GATE 2012 question pattern:** what does this print?

```
int j;
for (j = 0; j < 10; j++);
printf("%d", j);
```

Answer: 10. The semicolon on the for-line is the loop's entire body! j marches 0→9, the update makes it 10, condition fails, printf runs ONCE with j = 10. Remove that semicolon and it prints 0123456789. Read the for-line twice before answering any GATE loop question.

■ **WHAT GATE ASKS (patterns from 2007–2026 papers)**

- Switch fall-through pattern (appears with small variations):

```
int x = 2;
switch (x) {
case 1: printf("A");
case 2: printf("B");
case 3: printf("C"); break;
case 4: printf("D");
}
// prints "BC" — entering case 2, falling into 3, break stops it
```

Answer: BC. No break after case 2 → falls into case 3 → prints C → break exits. Case 4 is never touched.

1.5 Scope & storage — nested rooms and the elephant

Real life: scope is rooms inside rooms. Walk into the inner room and the outer room's poster becomes invisible — you only see the innermost version of the same name. And a **static** variable is an elephant: it never forgets its value between function visits. An ordinary (auto) local is a goldfish — born fresh, brain wiped every visit.

■ **THE BOOKISH CORE (what the textbook says)**

- **auto** (default locals): created on entry, destroyed on exit, UNINITIALISED garbage.
- **static** local: initialised ONCE (0 by default if you don't set it), value PERSISTS across calls. Also gives internal linkage for globals (invisible outside the file).
- **extern**: declares a variable defined in another file — "it exists, trust me, defined elsewhere".
- **Shadowing:** an inner block may reuse an outer name; inside the block only the inner one is visible. C has no :: escape hatch (that's C++).

■ **WHAT GATE ASKS (patterns from 2007–2026 papers)**

- **Shadowing pattern (GATE 2008 asked essentially this):**

```
int i = 10; // global room
int main() {
int i = 20; // middle room
{ int i = 30; // innermost room
printf("%d ", i); } // which i? -> 30
printf("%d", i); // and now? -> 20
} // global 10 never got a turn
```

Answer: 30 20. Each printf sees the nearest surrounding declaration. Rule: "innermost wins".

■ **WHAT GATE ASKS (patterns from 2007–2026 papers)**

- **The static counter (asked in many forms — 2005, 2014, 2017…):**

```
void counter() {
static int c = 0; // runs ONCE ever
c++;
printf("%d ", c);
}
int main() {
counter(); counter(); counter();
}
```

Answer: 1 2 3. The initialisation happens on the first call only; afterwards c simply remembers. With a plain **int c = 0;** it would print 1 1 1 — swap static for auto and you've flipped the answer; GATE tests whether you noticed.

1.6 Pointers — the chapter GATE secretly loves most

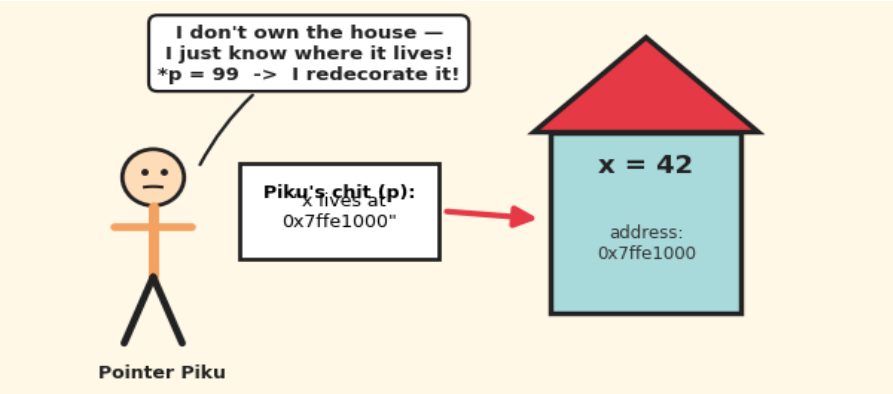


Fig 1.2 — A pointer doesn't hold the value; it holds the address where the value lives.

■ THE BOOKISH CORE (what the textbook says)

- **&x;** = "address of x". ***p** = "the thing at address p". Declaration **int *p = &x;** reads right-to-left as "p is a pointer to int, now aimed at x".
- ***p = 99 changes x.** That single sentence is why pointers exist: functions can modify their caller's variables by receiving addresses.
- **Pointer arithmetic scales by type:** if p is int* (4 bytes) at 1000, then p+1 is address 1004, p+2 is 1008. char* moves 1 byte per step — that's why char* walks strings byte by byte.
- **Array–pointer identity:** a[i] ≡ *(a+i) ≡ *(i+a) ≡ **i[a]**. The last form is legal C, not a typo!
- Reading complex declarations: **[] binds tighter than ***. **int *p[5]** = array of 5 int-pointers (size 5×8 = 40). **int (*p)[5]** = ONE pointer to an array of 5 ints (size 8). GATE asks their sizeof values.
- const maze, read backwards from the name: **const int *p** (p can move, *p can't change) vs **int * const p** (p fixed, *p changeable) vs **const int * const p** (both frozen).

■ WHAT GATE ASKS (patterns from 2007–2026 papers)

- **Pointer arithmetic classics (multiple GATE years):**

```
int a[] = {10, 20, 30, 40};
int *p = a + 2; // p points at a[2] = 30
printf("%d %d", p[-1], 2[a]);
```

Answer: 20 30. p[-1] means *(p-1) → a[1] = 20 — negative indices are legal and meaningful. 2[a] means *(2+a) → a[2] = 30. Yes, really. If GATE offers "compilation error" for 2[a], that option is bait.

■ WHAT GATE ASKS (patterns from 2007–2026 papers)

- **Pointer-to-pointer (asked 2004, 2017):**

```
int x = 10, *p = &x, **q = &p;
**q = 50;
printf("%d", x);
```

Answer: 50. Chase the arrows: q → p → x. **q is just x wearing two coats. Trace diagrams like Fig 1.2 and you'll never miss these.

■■ TRAP ALERT (where rank-hopefuls bleed marks)

- Uninitialised pointer ("wild") → segfault. NULL pointer → segfault. Dangling pointer (to a dead variable) → segfault that debuggers miss. GATE's favourite runtime-crash options.
- ***p++ means *(p++)** — move the pointer, use the old pointed value. **(*p)++** — increment the pointed value. One space of brackets, totally different answer.
- Thinking int *p[5] and int (*p)[5] are the same — sizeof alone kills that idea: 40 vs 8.

1.7 Arrays & strings — flats, parades and the hidden guard

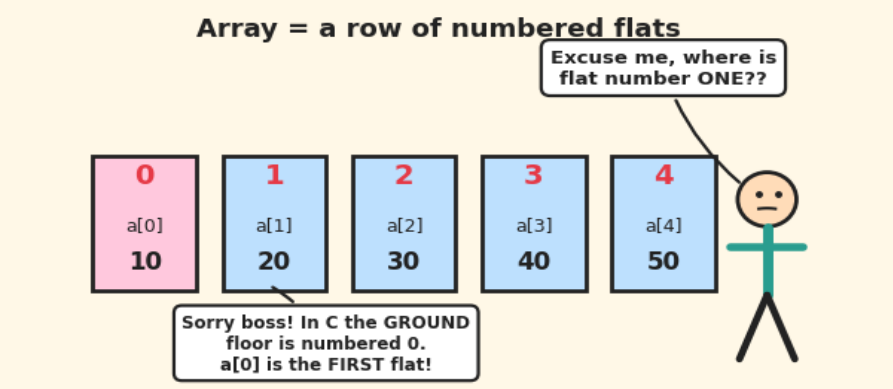


Fig 1.3 — a[0] is the FIRST element. GATE has punished "flat 1" thinking for two decades.

■ **THE BOOKISH CORE (what the textbook says)**

- **Partial initialisation zero-fills:** int a[5] = {1,2,3}; → 1 2 3 0 0. But int a[5]; then a is all garbage.
- **No bounds checking.** a[5] on a 5-element array reads the neighbour's locker — no error, just garbage/undefined. GATE assumes you know this and quietly walks you off the end.
- The array name "decays" to a pointer to its first element in MOST contexts — EXCEPT as operand of sizeof, & , or string literal initialiser. Hence **sizeof(array) = total bytes** (e.g., 5×4 = 20) while **sizeof(pointer) = 8** always.
- **2-D arrays are row-major:** int a[2][3] stores row 0 fully, then row 1: a[i][j] sits at base + (i*3 + j)*4 bytes. a[i] alone = pointer to row i.
- **Strings** = char arrays ending in '\0'. 'GATE' needs 5 bytes. strlen counts characters EXCLUDING '\0'. strcpy/strcat/strcmp live in <string.h>; **== on strings compares addresses, never content**.

■ **WHAT GATE ASKS (patterns from 2007–2026 papers)**

- **The single most repeated array/string question pattern:**

```
char s1[] = "HELLO";
char *s2 = "HELLO";
printf("%zu %zu %zu", sizeof(s1), sizeof(s2), strlen(s1));
```

Answer: 6 8 5. s1 is a true array of 6 chars (5 letters + \0) → 6. s2 is a pointer → 8 on 64-bit. strlen stops before \0 → 5. One more brutal follow-up GATE loves: **s1[0] = 'M';** is fine, but **s2[0] = 'M';** → **CRASH** — string literals live in read-only memory. Modifying one is the classic segfault question.

1.8 Functions & recursion — plumbers, dreams and the call stack

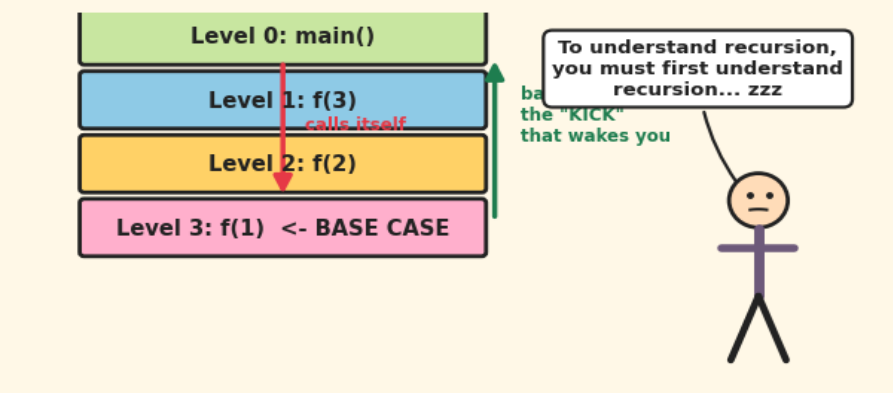


Fig 1.4 — Each recursive call is a new dream level; the base case is the "kick" that wakes you back up.

■ THE BOOKISH CORE (what the textbook says)

- C is strictly **call-by-value**: the function receives COPIES. To change the caller's variable, pass its address: **void f(int *p){ *p = 99; }** called as f(&x;).
- Every call creates a **stack frame** (local copy + return address) on the call stack. Frames pop as functions return — this memory model is what makes recursion work, and what kills it when it never stops.
- **Recursion needs two things:** (1) a base case that returns without calling again, (2) every call must move toward it. Missing base case → stack frames pile up forever → **stack overflow** crash (Inception's limbo).
- Where you print/act matters: work done BEFORE the recursive call happens on the way DOWN; work done AFTER happens on the way UP.

■ WHAT GATE ASKS (patterns from 2007–2026 papers)

- **Trace-it patterns (GATE asks output-tracing nearly every year):**

```
void f(int n) {
if (n == 0) return; // the "kick"
printf("%d ", n);
f(n - 1); // print BEFORE going deeper
}
f(3); // -> 3 2 1

void g(int n) {
if (n == 0) return;
g(n - 1); // go deeper FIRST
printf("%d ", n);
}
g(3); // -> 1 2 3 (same calls, opposite order!)
```

Answers: 3 2 1 and 1 2 3. Identical calls, mirror outputs. Draw the dream-levels of Fig 1.4, write prints next to each level: f prints while descending, g prints while ascending. Half of all GATE recursion questions are exactly this idea with extra decorations.

■ WHAT GATE ASKS (patterns from 2007–2026 papers)

- **Accumulator pattern:**

```
int sum(int n) {
if (n == 0) return 0;
return n + sum(n - 1);
}
printf("%d", sum(5));
```

Answer: 15 (5+4+3+2+1). Unwind from the deepest level: sum(0)=0 → sum(1)=1+0 → sum(2)=2+1 … → sum(5)=15. Practice unwinding bottom-up; top-down guessing is how 44,000-rank answers are born.

■■ TRAP ALERT (where rank-hopefuls bleed marks)

- Recursion with a static/global counter: the variable is SHARED across all dream levels — trace it once, it never resets (GATE 2005/2014 pattern).
- A function declared to return int that returns nothing on some path = undefined value — GATE's favourite "garbage value" answer.

1.9 Preprocessor & macros — the photocopy intern

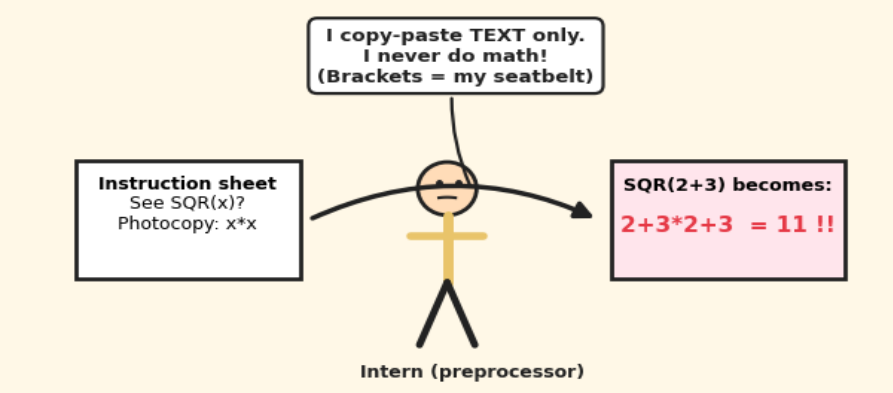


Fig 1.5 — Macros do blind text substitution. 2+3*2+3 = 11, not 25. Brackets are the seatbelt.

■ THE BOOKISH CORE (what the textbook says)

- #define does **pure text replacement before compilation**. The compiler never sees the macro name — it sees whatever text was pasted.
- Function-like macro without brackets: **#define SQR(x) x*x** → SQR(2+3) pastes as 2+3*2+3 = **11**, because * binds tighter than +. Fix: **#define SQR(x) ((x)*(x))** → 25.
- Macros evaluate arguments MULTIPLE times: **SQR(i++)** pastes ((i++)*(i++)) — increments twice, undefined behaviour. Functions evaluate once. (inline functions give you speed without this horror.)
- Multi-statement macros wrapped in **do { … } while(0)** so they behave like one statement everywhere.
- #include pastes files; #ifdef / #if compile code conditionally; header guards (#ifndef H / #define H … #endif) stop double-pasting of headers.

■ WHAT GATE ASKS (patterns from 2007–2026 papers)

- **Macro expansion output — asked 2010, 2013, 2018:** what does this print?

```
#define SQR(x) x * x
int main() {
printf("%d", SQR(2 + 3));
}
```

Answer: 11. The intern pastes: 2 + 3 * 2 + 3. Precedence does the rest: 2 + 6 + 3 = 11. The whole question is one sentence long: "the preprocessor cannot do mathematics." If your answer was 25, you calculated; the intern only photocopied.

1.10 Rapid-fire quiz — 10 questions, then check answers

```
1. printf("%zu %zu", sizeof('G'), sizeof("G"));
2. int i; for(i=0;i<8;i++); printf("%d",i);
3. Inner i=30/20 shadowing program of §1.5
4. counter() with static int c=0; c++; called 3×
5. a={10,20,30,40}; p=a+2; printf p[-1]
6. Same array: printf("%d", 2[a])
7. int x = (1, 2, 3);
8. #define SQR(x) x*x → SQR(2+3)
9. int i=0; printf("%d %d", 0 && i++, i);
10. f(3) prints n then calls f(n-1), base n==0
```

■ **ANSWERS (cover the right column, attempt first!)**

- **1.** 4 and 2 — constant is int (4); string adds \0 (2).
- **2.** 8 — the lonely semicolon is the loop body.
- **3.** 30 20 — innermost declaration wins.
- **4.** 1 2 3 — initialisation once, memory forever.
- **5.** 20 — negative index means *(p-1), totally legal.
- **6.** 30 — a[i]==*(a+i)==*(i+a)==i[a]. Yes, really.
- **7.** x = 3 — comma operator keeps only the last gift.
- **8.** 11 — 2+3*2+3. Brackets would give 25.
- **9.** 0 0 — && short-circuits, i++ never runs, i still 0.
- **10.** 3 2 1 — before-recursion work prints on the way down.

1.11 One-page cheat sheet — stick this on your wall

Topic	The one truth GATE keeps asking
Sizes	char 1 · short 2 · int 4 · long 8 · float 4 · double 8 · pointer 8 (64-bit). sizeof('A')=4, sizeof("A")=2.
++/--	a++ : use old value, then increment. ++a : increment, then use. Same name twice in one expression = UNDEFINED.
Division	5/2 = 2 (int). -7/2 = -3, -7%2 = -1 (truncate toward zero). Unsigned: 0-1 → 4294967295.
Comma	(a, b, c) → value c. Function-argument commas are NOT operators.
&& ||	Short-circuit: 0 && anything → right side never runs. Guards NULL before dereference.
Semicolon	for(...); → empty body, loop spins in place. Read the for-line twice.
switch	No break → fall through to next case. default = "the rest".
static	Local static: init once, remembers forever → 1 2 3 pattern. Global static: private to file.
Scope	Innermost declaration wins. Inner { int i } hides outer i. No :: rescue in C.
Pointer	&x; = address, *p = contents. p+1 jumps sizeof(type) bytes. *p++ ≠ (*p)++.
a[i]	a[i] = *(a+i) = *(i+a) = i[a]. Array decays to pointer EXCEPT in sizeof and &.
Array vs ptr	char s[]="HI" → 3 bytes, modifiable. char *s="HI" → 8-byte pointer, read-only crash if modified.
Strings	strlen skips \0, sizeof keeps it. Never == two strings; use strcmp.
Recursion	Base case = the kick. Work before call → prints descending; after call → ascending.
Macros	#define SQR(x) ((x)*(x)) or die: SQR(2+3)=11. Macros can't count; they paste text.
Call-by-value	Functions get copies. To edit caller's variable → pass &x;, take int *p, write *p.

How this chapter fits your 22-week plan

This is **Chapter 1 of ~19** covering the full GATE 2027 CS syllabus in this exact style. Suggested use with your evening 3-hour block: **Day 1** §1.1–1.4 (retype every code example by hand and RUN it), **Day 2** §1.5–1.7 + quiz, **Day 3** §1.8–1.11 + write the cheat sheet from memory. Then solve C PYQs 2007–2026 — you'll be shocked how familiar they look.

Ch	Topic	Status
1	C Programming (this chapter)	✔ DONE
2	DSA-I: Arrays, Strings, Recursion deep-dive	next
3	DSA-II: Linked Lists, Stacks, Queues	
4	DSA-III: Trees, BST, Heaps	
5	DSA-IV: Graphs — BFS/DFS, MST, Shortest Paths	
6	Algorithms: complexity, sorting, hashing, greedy	
7	Dynamic Programming (GATE's favourite child)	
8	Discrete Math: logic, proofs, induction	
9	Discrete Math: sets, relations, functions, lattices	
10	Combinatorics + counting	
11	Linear Algebra	
12	Probability & Statistics	
13	Digital Logic	
14	Computer Organization & Architecture	
15	Operating Systems	
16	DBMS	

17	Computer Networks	
18	Theory of Computation + Compiler Design	
19	General Aptitude crash course	

Say "next chapter" and I'll build Chapter 2 (DSA-I) in this same format. File every solved example from this chapter in your error notebook — each one becomes a flashcard.

GATE 2027 CS — Chapter 2: DSA-I: Arrays, Strings & Recursion, GATE-style

Contiguous memory, the two-pointer squeeze, binary search, minimum-comparison facts, pattern matching, and recursion you can actually trust.

Chapter 1 gave you the C language; this chapter turns it into a weapon. Roughly 10–12 of GATE's 100 marks sit on array/string/recursion questions — and they are the most predictable marks in the whole paper. Every pattern below appears almost every year.

2.1 The array — one long shelf of equal boxes

■ THE BOOKISH CORE (what the textbook says)

- An array is **contiguous** memory of same-type elements. Address of a[i] = base + i × sizeof(type). That single formula explains everything: O(1) random access, why indices start at 0, and why the array name is really just a pointer to the shelf.
- Costs: access O(1) · search O(n) unsorted / O(log n) sorted · insertion-deletion in middle O(n) (everyone shifts) · reversal O(n).

■ WHAT GATE ASKS (patterns from 2007–2026 papers)

- "The minimum number of interchanges needed to convert array A into array B?" → think in cycles: each k-cycle needs k−1 swaps. (GATE asked swaps to transform a permutation, 2008.)
- Finding an element in an UNSORTED array: always O(n) — no shortcuts exist. GATE's distractors love claiming O(log n) for unsorted.

■■ TRAP ALERT (where rank-hopefuls bleed marks)

- Accessing a[i] for i outside 0..n−1 is legal-looking code with undefined results — GATE expects you to catch it in loop-bound questions.
- Confusing the array's total size with element count: sizeof(a)/sizeof(a[0]) — forget this once and every loop in the exam feels haunted.

2.2 Two pointers & rotation — the train-bogie trick

Real life: a row of people sorted by height, and you must find two whose heights add to exactly the target. Do you compare everyone with everyone? No — you put one kid at each end. Too tall together? The tall end steps in. Too short? The short end steps in. Each step kills a whole row of impossible pairs. That is the **two-pointer technique**, and it only works because the line is sorted.

■ THE BOOKISH CORE (what the textbook says)

- **Pair-sum on sorted array:** L at 0, R at n−1. While L<R: if a[L]+a[R]==target → found; if sum < target → L++; else R--. O(n) time, O(1) space.
- **Rotate left by k** (train-bogie trick): reverse the first k elements, reverse the rest, reverse everything. O(n) time, O(1) extra space. Example below.
- Same squeeze removes duplicates from a sorted array in O(n) by writing survivors at a "slow pointer".

■ WHAT GATE ASKS (patterns from 2007–2026 papers)

- GATE repeatedly asks: rotate [1 2 3 4 5 6 7] left by 3 using O(1) extra space → reversal trick → [4 5 6 7 1 2 3]. The "obvious" O(k) shifting method is O(nk) — that contrast IS the question.

```
// rotate(a, n, k): reverse(a, 0, k-1); reverse(a, k, n-1); reverse(a, 0, n-1)
a = [1 2 3 4 5 6 7], k = 3
reverse first 3 : [3 2 1 | 4 5 6 7]
reverse rest : [3 2 1 | 7 6 5 4]
reverse all : [4 5 6 7 1 2 3] <- done, O(n), no extra array
```

2.3 Binary search — halving the haystack

■ **THE BOOKISH CORE (what the textbook says)**

• Needs a **sorted** array. Repeatedly compare target with a[mid]; discard half. Worst-case comparisons for n elements: ■**log2(n+1)**■. For n = 127 → 7 comparisons; n = 1000 → 10.

• Overflow-safe mid: **mid = lo + (hi** – **lo)/2** (lo+hi can overflow int). GATE has asked this as an MCQ.

• Variants: first occurrence (move hi on equality), last occurrence (move lo on equality), search in a rotated-sorted array (decide which half is sorted) — all O(log n).

• Binary search on ANSWER (not array): "minimum capacity to ship in D days" style — rare in GATE, common in interviews; know it exists.

■ **WHAT GATE ASKS (patterns from 2007–2026 papers)**

• GATE pattern: "sorted array of 1023 elements, worst case comparisons?" → ■log2(1024)■ = 10. Powers of two minus one are GATE's favourite numbers for this reason.

```
int bsearch(int a[], int n, int key) {
int lo = 0, hi = n - 1;
while (lo <= hi) {
int mid = lo + (hi - lo) / 2; // overflow-safe
if (a[mid] == key) return mid;
else if (a[mid] < key) lo = mid + 1;
else hi = mid - 1;
}
return -1; // not found -> log2(n) probes gone
}
```

2.4 Minimum comparisons — the tournament facts GATE memorises for you

Real life: finding the strongest player and the runner-up. If everyone plays everyone, that's n(n–1)/2 matches — insane. A knockout tournament finds the champion in **n** – **1** matches (everyone but the champion loses exactly once). And the runner-up must have lost TO the champion — they're hiding among the champion's opponents.

■ **THE BOOKISH CORE (what the textbook says)**

• **Min (or max) alone: n** – **1 comparisons.** Each comparison eliminates exactly one candidate.

• **Both min AND max together: ■3n/2■** – **2 comparisons** — process elements in pairs, compare the pair (1 comparison), then the winner challenges current max and the loser challenges current min. Halves the work vs 2(n–1). GATE asks this exact number.

• **Second largest: n + ■lg n■** – **2.** Run the knockout (n–1 matches), then only the ■lg n■ players the champion defeated can be runner-up: find their max in ■lg n■ – 1 more.

■ **WHAT GATE ASKS (patterns from 2007–2026 papers)**

• "The minimum number of comparisons required to find both the minimum and the maximum of 1000 elements" → ■3n/2■ – 2 = 1498. (This was a genuine GATE question — the options are designed around 2n–2 = 1998, the naive answer.)

2.5 Strings & pattern matching — finding a word in a paragraph

■ THE BOOKISH CORE (what the textbook says)

- **Brute force:** slide the pattern over the text, comparing character by character. Worst case **O(m·n)** (e.g., text "AAAA...A", pattern "AAAB"). GATE asks this worst case with a concrete example.
- **KMP** fixes the re-checking: precompute the **prefix (failure) function** π for the pattern — π[i] = length of the longest proper prefix of pattern[0..i] that is also a suffix. On mismatch, slide by the safe amount instead of restarting.
- KMP cost: O(m + n) comparisons — GATE asks the table and the complexity.
- Computing π is itself a mini-recursion: it reuses π values computed earlier ("the table builds itself").

■ WHAT GATE ASKS (patterns from 2007–2026 papers)

- GATE pattern: "pattern = ABABCABAB, build the failure function" → π = [0,0,1,2,0,1,2,3,4]. Practice building this table by hand until it takes 60 seconds.

```
pattern: A B A B C A B A B
pi : 0 0 1 2 0 1 2 3 4
# pi[4]=0 because "ABABC" has no proper prefix==suffix
# pi[8]=4 because "ABAB" is both prefix and suffix of pattern[0..8]
```

■■ TRAP ALERT (where rank-hopefuls bleed marks)

- After a full match, KMP continues with π[last] — it does NOT restart from the next character. GATE's "how many comparisons" questions punish restart-thinking.

2.6 Recursion deep-dive — the leap of faith, with numbers

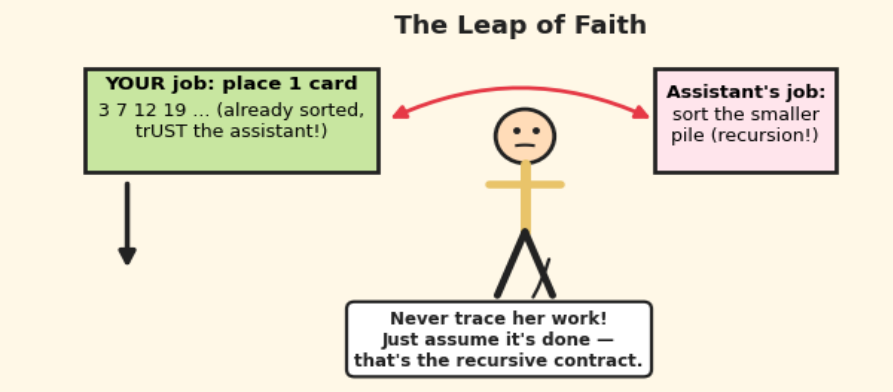


Fig 2.1 — The recursive contract: trust the assistant on the smaller problem; you handle only your piece.

■ THE BOOKISH CORE (what the textbook says)

- Writing recursion = two lines of discipline: (1) **base case** first — the smallest input you answer directly; (2) **recursive case** — solve the problem assuming the smaller version is already solved. Never think about HOW the assistant works; that way madness lies.
- Counting recursive calls: draw the recursion tree; each node is one call. A full binary recursion tree of depth d has 2^(d+1) – 1 nodes — GATE asks this count.
- Recursion + static/global variables: ONE shared variable across every level — trace it top-down once, and remember Chapter 1's elephant.

■ WHAT GATE ASKS (patterns from 2007–2026 papers)

- **A genuine GATE 2017 question — attempt before reading the answer:**

```
int f(int n) {
static int r = 0; // the elephant returns!
if (n <= 0) return 1;
if (n > 3) { r = n; return f(n - 2) + 2; }
return f(n - 1) + r;
}
printf("%d", f(5));
```

Answer: 18. Trace: f(5) sets r=5, returns f(3)+2. f(3) = f(2)+r = f(2)+5. f(2) = f(1)+5. f(1) = f(0)+5 = 1+5 = 6. Unwind: f(2) = 11, f(3) = 16, f(5) = 16+2 = **18**. Three skills in one 4-line question: static memory, conditional recursion, bottom-up unwinding. This is why this pattern repeats.

■ **WHAT GATE ASKS (patterns from 2007–2026 papers)**

- Print-an-array-in-reverse pattern (asked 2014, 2019): recursion BEFORE the print → reverse order for free.

```
void rev(int a[], int n) {
if (n == 0) return;
rev(a, n - 1); // go deep first...
printf("%d ", a[n - 1]); // ...print on the way up
}
rev([1 2 3 4], 4) -> 4 3 2 1
```

2.7 Loop complexity — read the shape, know the answer

■ **THE BOOKISH CORE (what the textbook says)**

- for(i=0;i1){m/=2;} → O(log m) same as the *=2 loop

■■ **TRAP ALERT (where rank-hopefuls bleed marks)**

- A loop body with only O(1) work vs one with an inner O(n) call — multiply everything. GATE hides printf-free inner calls to test whether you count work, not lines.

Rapid-fire quiz — 10 questions, then check answers

```
1. Sorted array [2 4 7 9 11 15]. Pair summing to 16? Two-pointer steps?
2. [1 2 3 4 5 6 7 8], rotate left by 3, O(1) space?
3. Binary search worst case on 2047 elements?
4. Min AND max of 500 elements, min comparisons?
5. Second largest via tournament, n = 32?
6. Brute-force pattern matching worst case?
7. KMP comparisons for pattern+text?
8. The f(5) static-recursion GATE 2017 program
9. rev(a,4) prints n-1 after recursive call → output?
10. Complexity: for(i=1;i<=n;i*=2){for(j=0;j<n;j++)work();}
```

■ **ANSWERS (cover this box, attempt first!)**

- **1.** Yes: 7+9. O(n); each step eliminates a whole row of pairs.
- **2.** Reversal trick → [4 5 6 7 8 1 2 3].
- **3.** ■log2(2048)■ = 11 comparisons.
- **4.** ■3n/2■ – 2 = 748.
- **5.** n + ■lg n■ – 2 = 32 + 5 – 2 = 35.
- **6.** O(m·n) — text "AAA...A", pattern "AAAB".
- **7.** O(m + n) — the failure function removes re-checks.
- **8.** 18 — trace it: 6, 11, 16, then +2.
- **9.** 4 3 2 1 — prints on the way up.
- **10.** O(n log n) — the classic.

One-page cheat sheet — stick this on your wall

Topic	The one truth GATE keeps asking
Array address	a[i] = base + i×sizeof(type). Index 0 is just base + 0.
Rotation	Left by k: reverse(0,k-1), reverse(k,n-1), reverse(0,n-1). O(n), O(1) space.
Two pointers	Sorted + pair-sum → squeeze from both ends. O(n). Never works on unsorted.
Binary search	Needs sorted. ■log2(n+1)■ worst-case probes. mid = lo + (hi-lo)/2.
Tournament	Max: n–1. Min+max: ■3n/2■–2. 2nd max: n+■lg n■–2.
KMP	Failure table π first, then O(m+n) scan. π = longest prefix=suffix lengths.
Leap of faith	Assume smaller case solved. You only: base case + combine step.

Topic	The one truth GATE keeps asking
Calls count	Recursion tree nodes = calls. Full tree depth d → 2^(d+1) – 1 calls.
static + recursion	One shared variable across all levels — trace once, it never resets.
Loop shapes	n / log n / n² / n log n — read the loop, name the shape in 2 seconds.

Say "next chapter" for Chapter 3: Linked Lists, Stacks & Queues — the pointer-dance chapter where GATE gives away 4–6 marks a year.

GATE 2027 CS — Chapter 3: DSA-II: Linked Lists, Stacks & Queues

The pointer-dance chapter. Node surgery, cycle detection, the stack/queue machines GATE has asked in 10+ different disguises.

GATE hands out 4–6 marks a year here, and the questions are glorified pattern-matching: read the pointer surgery, know what falls out. This chapter teaches you to simulate code with a pen instead of guessing.

3.1 Linked list — the treasure hunt data structure

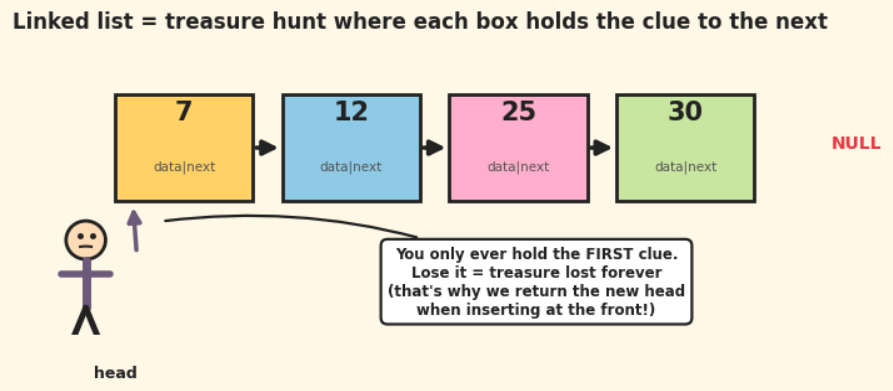


Fig 3.1 — head is your only entry. Every box stores data + the address of the next box.

■ THE BOOKISH CORE (what the textbook says)

- A node = **data + pointer to next node**. The list is accessed only through **head**. Unlike arrays: dynamic size, O(1) insert/delete if you already have the pointer to the spot — but **O(n) search and no random access** (walk the chain).
- Types GATE names: singly (next only), doubly (prev+next, O(1) delete given node), circular (last points to first; useful for round-robin).
- Costs vs array: search O(n) vs O(n)/O(log n); insert-front O(1) vs O(n); insert-middle O(1) with pointer vs O(n). GATE asks you to pick the structure from the operation list.

■ WHAT GATE ASKS (patterns from 2007–2026 papers)

- "Which data structure for frequent insertions at both ends?" → doubly linked list (or deque). "Frequent binary search?" → sorted array, never a list.

■■ TRAP ALERT (where rank-hopefuls bleed marks)

- Dereferencing NULL (walking past the last node) = crash. Every while loop must check p != NULL BEFORE p->next. GATE's segmentation-fault options are counting on sloppy boundary reading.

3.2 The pointer dance — four surgeries you must know cold

Real life: a single-linked list is a conga line where each person only holds the next person's hand. To remove someone, you don't touch them — the person BEFORE them just grabs the next-next hand. Surgery happens on the link, not the patient.

■ THE BOOKISH CORE (what the textbook says)

- **Insert at front** (the one that changes head!): new->next = head; head = new;
- **Delete after node q**: p = q->next; q->next = p->next; free(p); — the victim is unlinked first, memory freed after. Never free before relinking.
- **Reverse (3 pointers):** prev=NULL, cur=head; while(cur){ nxt=cur->next; cur->next=prev; prev=cur; cur=nxt; } head=prev. O(n).
- **Slow-fast pointers:** slow moves 1, fast moves 2. Fast hits NULL → no cycle; they meet → cycle exists. Meeting is guaranteed inside the loop because fast gains 1 per lap.

■ WHAT GATE ASKS (patterns from 2007–2026 papers)

- GATE's favourite disguise: "What does mystery(head) print?" with exactly the 4-line reverse-print recursion:

```
void mystery(struct node *p) {
if (p == NULL) return;
mystery(p->next); // dive to the end first
printf("%d ", p->data); // print while surfacing
}
// called on 7 -> 12 -> 25 -> 30 : prints 30 25 12 7
// = recursion print-on-the-way-up, wearing a linked-list costume.
```

Answer: the list in reverse. Same soul as Chapter 2's rev() function. When you spot "recursion before the action", answer "reversed". GATE asked this pattern in 2014 and again in 2019 — it will appear again.

■ WHAT GATE ASKS (patterns from 2007–2026 papers)

- **Cycle detection (asked in GATE 2013):** "The following function checks for a loop in a linked list using two pointers." Know the answer before reading options: slow-fast meeting proves a cycle; if fast or fast->next becomes NULL, list is acyclic.

■■ TRAP ALERT (where rank-hopefuls bleed marks)

- In the reverse surgery, losing nxt before using it strands the rest of the list — GATE shows 3-pointer code with one line swapped; simulate on 3 nodes A→B→C and the error confesses.
- Insert-after on the LAST node without updating tail (in a list that maintains tail) — concept questions about where tail points after operations.

3.3 Stack — the plate pile (LIFO)

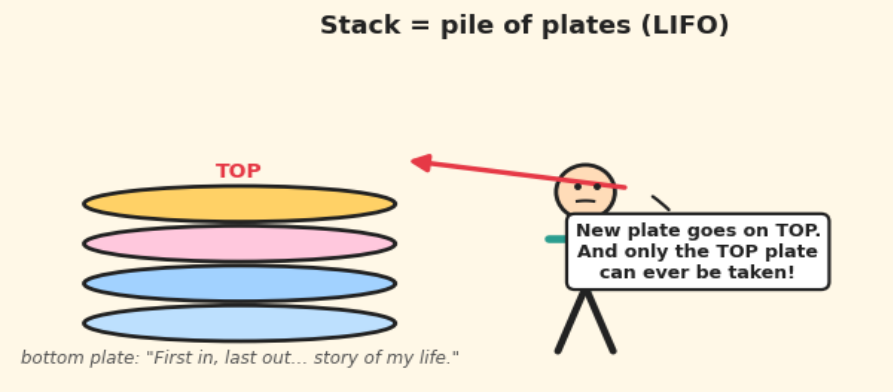


Fig 3.2 — Push and pop both happen at the top. Bottom plate waits the longest.

■ THE BOOKISH CORE (what the textbook says)

- push, pop, top, isEmpty — all O(1). The LAST element pushed is the FIRST out (LIFO). Underflow = pop from empty; overflow = push to a full fixed array.
- Array implementation: top starts at –1; push does a[++top] = x. Linked implementation: push at head, O(1), grows forever.
- Four applications GATE rotates through: balanced parentheses, infix→postfix conversion, postfix evaluation, function-call stack (you met it in Ch. 1).

■ WHAT GATE ASKS (patterns from 2007–2026 papers)

- **Balanced parentheses (GATE asks variations):** scan left to right; on "(" push; on ")" pop and match. Stack empty at a ")" → unbalanced; non-empty at end → unbalanced. O(n), one pass.

```
stack: ( ( ) ( ) ) -> push,push,pop,push,pop,pop -> ends empty -> BALANCED
stack: ( ) ) -> the third char pops from EMPTY -> UNBALANCED
```

■ WHAT GATE ASKS (patterns from 2007–2026 papers)

- **Infix → Postfix:** operands print immediately; "(" push; ")" pops till "("; operators pop while top has ≥ precedence, then push. Postfix has NO brackets — the order encodes the tree. GATE gives a messy expression and asks for the postfix (or the value after evaluation).

```
infix : A + B * C + D precedence: * > +
postfix: A B C * + D + (multiply binds first, left-to-right for equal +
value of 2 3 4 * + 5 + : 2 + 12 + 5 = 19 (scan: numbers push, operator pops 2, computes, pushes
back)
```

■ WHAT GATE ASKS (patterns from 2007–2026 papers)

• **Stack permutations — a genuine GATE favourite:** input 1,2,3 pushed in that order; which outputs are possible? 123 ✓ 132 ✓ 213 ✓ 231 ✓ 321 ✓ — but **312 ✗** (3 must be popped before 2 arrives, but then 1 is buried under 2). Count of possible outputs for input 1..n = the Catalan number C_n: C_3 = 5, C_4 = 14, C_5 = 42 — GATE has asked all three values.

■■ TRAP ALERT (where rank-hopefuls bleed marks)

• Equal-precedence associativity: a – b – c = (a–b)–c, so in postfix conversion equal precedence STILL pops. Forgetting this flips answers.

• "Minimum stack space to evaluate postfix" questions: track the running size of the operand stack — the peak is the answer.

3.4 Queue — the ticket line (FIFO)

Real life: the line at a chai stall. New people join at the REAR, service happens at the FRONT. Nobody jumps, nobody is served twice. First in, first out — fairness as a data structure.

■ THE BOOKISH CORE (what the textbook says)

- enqueue (rear), dequeue (front), front, isEmpty — O(1) each. Applications: BFS (Chapter 5), scheduling, buffering.
- **Circular array implementation:** indices wrap with modulo n. front = (front+1)%n on dequeue; rear = (rear+1)%n on enqueue. Number of elements = (rear – front + n) % n when using the "waste one slot" convention: full when (rear+1)%n == front; empty when front == rear.
- **Queue with two stacks:** enqueue pushes to stack S1 (O(1)). Dequeue: if S2 empty, pop all of S1 into S2 (reverses order), then pop S2. Each element crosses each stack once → every operation is **amortized O(1)**. Worst-case single dequeue is O(n) though.

■ WHAT GATE ASKS (patterns from 2007–2026 papers)

- GATE 2006 asked exactly this: implement a queue using two stacks; the correct complexity claim is "enqueue O(1), dequeue amortized O(1)". The distractor "every operation worst-case O(1)" is the trap.

```
enqueue: push to S1
dequeue: if S2 empty { while(S1) push(S2, pop(S1)); } // once per element, ever
return pop(S2)
amortized: n enqueues + n dequeues cost ~4n stack ops -> O(1) per op
```

3.5 Deque & the border with heaps

■ THE BOOKISH CORE (what the textbook says)

- **Deque** (double-ended queue): insert/remove at BOTH ends O(1) — implemented with a circular array or doubly linked list. Input-restricted and output-restricted variants exist; GATE asks which structure suits which access pattern.
- **Priority queue**: dequeue removes the PRIORITY element, not the oldest — implemented with a heap (Chapter 4). A queue won't do; a heap isn't fair; know the difference.

■■ TRAP ALERT (where rank-hopefuls bleed marks)

- Mixing front/rear update ORDER in circular queue code — simulate on paper with n=4 and 5 enqueues; the %n must happen on the SAME line as the increment.

Rapid-fire quiz — 10 questions, then check answers

```
1. Node layout of a singly linked list?
2. Cost of inserting at front of a singly linked list?
3. mystery(p): recurse p->next, then print data → prints?
4. Slow-fast pointers meet → what does it prove?
5. Reverse (A B C) with prev/cur/nxt → result?
6. "(()))(()(" balanced?
7. Postfix of A + B * C, and its evaluation for 2 3 4 * +?
8. Possible stack outputs from input 1,2,3? Count?
9. Circular queue, n slots, "waste-one-slot" — full when?
10. Queue via two stacks — dequeue complexity?
```

■ **ANSWERS (cover this box, attempt first!)**

- **1.** data + one pointer (next). Doubly: data + prev + next.
- **2.** O(1): new->next = head; head = new. (Array: O(n).)
- **3.** List in reverse (30 25 12 7).
- **4.** The list has a cycle (Floyd's algorithm).
- **5.** C B A; head must be updated to old tail.
- **6.** No — the 7th char ")" pops from empty.
- **7.** A B C * +; value 2+12 = 14.
- **8.** 123,132,213,231,321 — five = Catalan C3. 312 impossible.
- **9.** (rear+1) % n == front; empty when front == rear.
- **10.** Amortized O(1); worst-case O(n) when S2 must be refilled.

One-page cheat sheet — stick this on your wall

Topic	The one truth GATE keeps asking
LL vs array	LL: O(1) insert/delete at known node, O(n) search, no random access. Array: opposite trade.
Head discipline	Only entry is head. Front-insert returns new head. Losing head = losing list.
Reverse LL	prev=NULL, cur=head; loop: nxt=cur->next; cur->next=prev; shift all three.
Floyd cycle	slow=1 step, fast=2; meet → cycle; NULL → none.
Stack apps	Parentheses, infix→postfix, postfix eval, calls. All one pass O(n).
Postfix eval	Numbers push; operator pops 2, computes, pushes. Final stack top = answer.
Stack perms	Input 1..n through one stack → Catalan C_n outputs. 312 is the classic impossible one.
Circular queue	rear=(rear+1)%n; full iff (rear+1)%n==front (wasted-slot convention).
2-stack queue	Enqueue S1 O(1); dequeue via S2 refill — amortized O(1) (GATE 2006).
Deque / PQueue	Deque: both ends O(1). Priority queue = heap, dequeues by priority not arrival.

Say "next chapter" for Chapter 4: Trees, BST & Heaps — the most-marks-per-page chapter in all of GATE CS.

GATE 2027 CS — Chapter 4: DSA-III: Trees, BST & Heaps

GATE's highest-yield DSA zone: 8–10 marks most years. Tree counting facts, the four traversals, BST discipline, and heap array tricks.

Everything in this chapter reduces to small integer facts plus one habit: **draw the tree**. A two-second sketch turns every question here into reading comprehension.

4.1 Tree arithmetic — the facts GATE recycles forever

■ THE BOOKISH CORE (what the textbook says)

- Level of root = 0. Height of tree = number of EDGES on the longest root-to-leaf path (a single node has height 0).
- A binary tree has **at most 2^i nodes at level i**; at most **2^(h+1) – 1** nodes total at height h. Minimum height for n nodes: **■lg(n+1)■ – 1**.
- **n0 = n2 + 1** — in ANY binary tree, leaves = (nodes with two children) + 1. Proof sketch: count edges two ways; every node except root borrows exactly one edge.
- **Strict (full)** binary tree: every node has 0 or 2 children. **Complete** binary tree: all levels full except possibly the last, filled left-to-right — this is the heap's body plan.
- A complete binary tree with n nodes has height **■lg n■**, and leaves sit at indices ■n/2■+1 .. n in the array layout.

■ WHAT GATE ASKS (patterns from 2007–2026 papers)

- "A binary tree has 15 leaves. How many nodes have exactly 2 children?" → 14, instantly, via n0 = n2 + 1. "A complete binary tree with 8 nodes has height?" → ■lg 8■ = 3. These are 30-second marks.

■■ TRAP ALERT (where rank-hopefuls bleed marks)

- Height counted in NODES vs EDGES is a classic 1-off dispute — GATE options are spaced to punish the wrong convention. Read the question's own definition; if none, edges is standard.
- In a tree with n nodes there are exactly n–1 edges — this single line answers a surprising number of GATE counting questions.

4.2 Traversals — four walks around the same tree

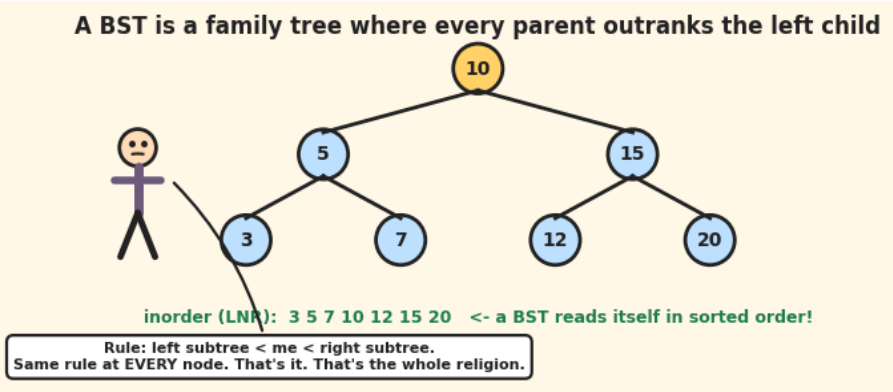


Fig 4.1 — A BST. Inorder (Left, Node, Right) always prints sorted.

■ THE BOOKISH CORE (what the textbook says)

- **Preorder (NLR)**: node, left, right — used to COPY a tree. **Inorder (LNR)**: left, node, right — on a BST, outputs sorted order. **Postorder (LRN)**: left, right, node — used to DELETE a tree (free children before parent). **Level-order**: BFS with a queue.
- **Reconstruction rule GATE worships:** preorder + inorder ⇒ UNIQUE tree. postorder + inorder ⇒ UNIQUE. preorder + postorder ⇒ NOT unique (need to know which nodes are leaves).
- Expression trees: inorder = infix (needs brackets), postorder = postfix = the exact order a stack machine evaluates. So "postfix of expression" = postorder traversal.

■ **WHAT GATE ASKS (patterns from 2007–2026 papers)**

• GATE pattern: "The preorder of a tree is ABCDEF and inorder is CBAEDF. Construct the tree." Method: first preorder letter = root; split inorder around it; recurse. Practice until a 6-node tree takes 60 seconds.

```
preorder : 10 5 3 7 15 12 20
inorder : 3 5 7 10 12 15 20
postorder: 3 7 5 12 20 15 10
level : 10 5 15 3 7 12 20 <- same tree, Fig 4.1
```

■■ **TRAP ALERT (where rank-hopefuls bleed marks)**

• Assuming preorder+postorder is enough — it's not. GATE explicitly tested this; only inorder pins down left-vs-right membership.

4.3 BST — search discipline

■ **THE BOOKISH CORE (what the textbook says)**

• Property: for every node, **all left descendants < node < all right descendants**. Searching: go left if key smaller, right if bigger — **O(h)**.

• Height h ranges **■lg(n+1)■ – 1 (balanced) to n – 1 (a chain)**. That's why sorted insertion (1,2,3,4,5) is the nightmare case — the BST degenerates into a linked list.

• **Insert** = unsuccessful search + hang new node at the dead end. **Min** = keep-leftmost. **Successor** = min of right subtree (if right exists), else lowest ancestor where you took a left turn.

• **Delete** three cases: leaf → snip; one child → bypass; two children → replace with inorder successor (or predecessor), then delete that successor from the subtree.

■ **WHAT GATE ASKS (patterns from 2007–2026 papers)**

• GATE pattern: "Which traversal of a BST prints keys in descending order?" → **Reverse inorder (RNL)**. And: "minimum number of comparisons to find 45 in this BST" → walk the path, count nodes.

• "Insert 50, 20, 70, 10, 30 into an empty BST" → draw it; then inorder = 10 20 30 50 70. GATE often asks the resulting inorder, height, or the search-path comparison count.

```
insert: 50, 20, 70, 10, 30, 60, 80
50
/ \
20 70
/ \ / \
10 30 60 80
search(60): 50 -> 70 -> 60 = 3 comparisons
```

■■ **TRAP ALERT (where rank-hopefuls bleed marks)**

• Equal keys: standard BSTs park duplicates to the right (or reject them) — read the question's convention before answering "where does the second copy go".

4.4 Heap — the champion's pyramid (array in disguise)

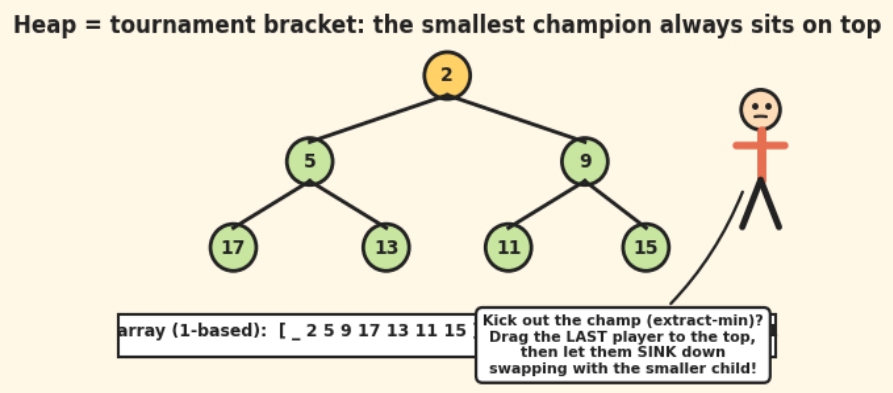


Fig 4.2 — Min-heap. Complete tree shape; array layout parent=■i/2■, children 2i and 2i+1 (1-based).

■ THE BOOKISH CORE (what the textbook says)

- **Min-heap:** every parent ≤ its children. The MINIMUM is always the ROOT. It is a **partial** order — siblings are unordered, so only the root is guaranteed.
- **Insert:** place at the next free array slot (keeps completeness), then **bubble UP**, swapping with a smaller parent — O(log n).
- **Extract-min:** move the LAST element to the root (fills the hole, keeps completeness), then **sift DOWN**, swapping with the SMALLER child — O(log n).
- **Build-heap from an array: O(n), not O(n log n)!** Half the nodes are leaves (already heaps); sift-down only from level ■lg n■–1 upward: n/4 nodes move ≤1, n/8 move ≤2, … total = O(n). GATE asks this number almost every cycle.
- **Heapsort:** build-heap O(n) + n extract-mins O(n log n) = **O(n log n)**, in-place, NOT stable. Delete-max variant sorts ascending.

■ WHAT GATE ASKS (patterns from 2007–2026 papers)

- GATE simulation pattern: "insert 7, 3, 9, 2, 5 into an empty min-heap, then delete-min twice — final heap?" Draw the array and simulate both operations; the answer is a 5-element array. There is no shortcut — but it takes 90 seconds if you know the two bubble rules.

```
heap ops (min-heap, 1-based array):
insert(7,3,9,2,5): [_,7,3,9,2,5] -> fix: [_,2,3,7,9,5]
delete-min x2 : remove 2 -> [_,7,3,5,9] -> sift -> [_,3,7,5,9]
remove 3 -> [_,9,7,5] -> sift -> [_,5,7,9]
```

■ WHAT GATE ASKS (patterns from 2007–2026 papers)

- **Where can the MAXIMUM hide in a min-heap?** Only among the LEAVES (indices ■n/2■+1 .. n). GATE asks this exact sentence; in the array it's the last ■n/2■ slots.

■■ TRAP ALERT (where rank-hopefuls bleed marks)

- During sift-DOWN you must compare BOTH children and swap with the SMALLER — swapping with the wrong child breaks the heap, and GATE's "which arrays are valid heaps" options exploit exactly this.
- Heap ≠ sorted array. [1, 5, 2, 9] can be a perfect min-heap (1≤5, 1≤2) yet look unsorted. Don't reject arrays for being unsorted — verify the parent-child inequalities only.

4.5 AVL — the self-balancing watchdog (short but real)

■ THE BOOKISH CORE (what the textbook says)

- AVL: a BST where every node's **balance factor (height-left** − **height-right)** ∈ **{–1, 0, +1}**. This keeps height O(log n) guaranteed.
- One insert/delete can unbalance exactly ONE node on the search path — fix with ONE rotation: LL → right rotate; RR → left rotate; LR → left then right; RL → right then left. GATE shows a small tree and asks which rotation restores balance.

■ WHAT GATE ASKS (patterns from 2007–2026 papers)

- Pattern: "Insert 30 into this AVL tree; which single rotation is performed?" Trace the path, find the deepest unbalanced ancestor, classify the grandchild pattern (LL/RR/LR/RL).

Rapid-fire quiz — 10 questions, then check answers

```
1. Tree with 20 leaves → how many degree-2 nodes?
2. Complete binary tree, 16 nodes → height?
3. Which traversal sorts a BST?
4. Which two traversals give a UNIQUE tree?
5. Postorder of an expression tree = ?
6. BST from inserting 10,5,15,3,7 — search(3) comparisons?
7. Worst BST height for n nodes? Best?
8. Min-heap insert cost? extract-min? build-heap?
9. In a min-heap array, where can the max be?
10. Heapsort: total complexity, in-place, stable?
```

■ ANSWERS (cover this box, attempt first!)

- **1.** 19 (n0 = n2 + 1).
- **2.** ■lg 16■ = 4.
- **3.** Inorder (LNR). Descending: reverse inorder.
- **4.** Pre+inorder, or post+inorder. Pre+post does NOT.
- **5.** The postfix form — stack-machine order.
- **6.** 3: 10 → 5 → 3.
- **7.** n–1 (chain, sorted input); ■lg(n+1)■–1.
- **8.** O(log n), O(log n), O(n) — the build is the exam trick.
- **9.** Only at leaves: indices ■n/2■+1 .. n.
- **10.** O(n log n), in-place, NOT stable.

One-page cheat sheet — stick this on your wall

Topic	The one truth GATE keeps asking
Tree facts	2^i max at level i; 2^(h+1)–1 total; n0 = n2 + 1; edges = n – 1.
Traversals	NLR / LNR / LRN / level. Inorder+pre ⇒ unique tree. Postorder deletes, preorder copies.
BST costs	Search-insert-delete O(h); h ∈ [■lg(n+1)■–1, n–1]. Inorder = sorted.
BST delete	Leaf → snip. One child → bypass. Two children → inorder successor replaces.
Heap shape	Complete tree; array 1-based: parent ■i/2■, kids 2i, 2i+1. Leaves from ■n/2■+1.
Heap ops	Insert: append + bubble UP. Extract: last→root + sift DOWN via smaller child.
Build-heap	O(n). Half the nodes are leaves; sift down from level lg n –1. Asked in GATE often.
Heap vs sorted	Heap is partial order: root guaranteed, children unordered. Verify parent≤child only.
AVL	Balance factor ∈ {–1,0,1}; one bad node per update; LL/RR/LR/RL rotations.
When in doubt	DRAW IT. Every tree question in GATE is 60 seconds with a sketch.

Say "next chapter" for Chapter 5: Graphs — BFS, DFS, MST and shortest paths, where GATE loves hiding 6 easy marks.

GATE 2027 CS — Chapter 5: DSA-IV: Graphs — BFS, DFS, MST & Shortest Paths

Syllabus §4(graphs) + §5(graph traversals, MST, shortest paths). 6–8 marks most years: BFS/DFS orders, Dijkstra traces, MST facts.

One habit carries this entire chapter: **keep a visited set and a distance label on every node**. Every GATE graph question is a 3–6 node toy graph; simulate the algorithm with a pen and the marks are yours.

5.1 Vocabulary & counting — the marks that take 20 seconds

■ THE BOOKISH CORE (what the textbook says)

- Simple undirected graph: max edges **n(n–1)/2** (complete graph K_n). Directed: max **n(n–1)**.
- **Handshaking lemma:** Σ degrees = 2E (undirected); Σ in-degree = Σ out-degree = E (directed). Odd-degree vertices are always EVEN in number.
- **Tree** = connected + acyclic ⇔ n vertices and n–1 edges. A forest with c components has n–c edges.
- **Euler circuit** (uses every edge once, returns): connected + every vertex even degree. **Hamiltonian cycle** (visits every vertex once): no simple criterion — NP-complete — but in K_n there are **(n–1)!/2** distinct ones.

■ WHAT GATE ASKS (patterns from 2007–2026 papers)

- **From IIT Madras's own GATE 2019 paper:** "Let G be an undirected complete graph on n vertices, n > 2. The number of different Hamiltonian cycles in G is…" → **(n–1)!/2**. Fix vertex 1, permute the other n–1 around the circle (n–1)! arrangements, divide by 2 because each cycle reads the same forwards and backwards. Madras loves exact counting — expect another one in 2027.

■■ TRAP ALERT (where rank-hopefuls bleed marks)

- Counting "paths between two vertices" by multiplying degrees — valid only in specific tree/complete-graph patterns; in general it's a DFS counting problem. GATE's attractive wrong answers come from degree multiplication.

5.2 Two ways to store a graph

Representation	Space	Edge lookup	Best for	GATE usage
Adjacency matrix	$O(V^2)$	O(1)	Dense graphs ($E \approx V^2$), Floyd-Warshall	"Is (u,v) an edge?" instant
Adjacency list	O(V + E)	O(degree)	Sparse graphs ($E \approx V$), BFS/DFS/Dijkstra	The default GATE structure

Rule of thumb: GATE's toy graphs are small enough to store either way; the question is what the ALGORITHM needs — iterating over all neighbours (list wins) or testing arbitrary pairs (matrix wins).

5.3 BFS — the ink drop (level = distance)

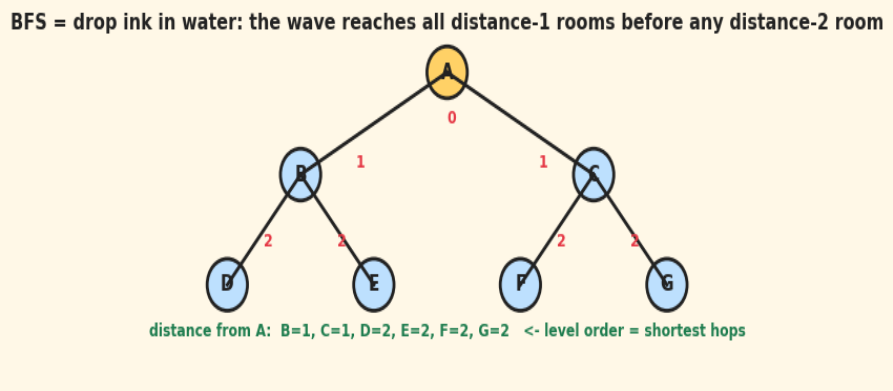


Fig 5.1 — BFS from A: the wave visits every distance-1 vertex before any distance-2 vertex.

■ **THE BOOKISH CORE (what the textbook says)**

- BFS uses a **QUEUE**: dequeue u, enqueue unvisited neighbours, mark visited. O(V + E).
- **On unweighted graphs, BFS order IS shortest-path distance.** That is the single most tested fact about BFS.
- BFS tree = the union of edges by which vertices were first discovered. Different neighbour orderings → different BFS trees, but the same distances.
- BFS 2-coloring tests bipartiteness: if a vertex ever meets an already-colored neighbour of its own color → odd cycle → NOT bipartite.

■ **WHAT GATE ASKS (patterns from 2007–2026 papers)**

- GATE pattern: "The BFS traversal of this graph starting at S, visiting neighbours in alphabetical order, is…" → simulate: queue contents step by step. 2 marks for 90 seconds of queue bookkeeping.
- "Which vertices are at distance exactly 2 from S?" → run BFS levels; count them. GATE asks this nearly every cycle.

```
BFS(S):
dist[S]=0; queue={S}
while queue not empty:
u = dequeue()
for each neighbour v of u (in given order!):
if v unvisited: dist[v]=dist[u]+1; enqueue(v)
```

5.4 DFS — the maze explorer

Real life: DFS is exploring a cave with a ball of string. You unroll the string going deeper and deeper down one tunnel; only when a tunnel dead-ends do you rewind to the last junction and try the next opening. The string is the **recursion stack**.

■ **THE BOOKISH CORE (what the textbook says)**

- DFS uses a **STACK** (explicit, or the call stack via recursion). O(V + E). Goes DEEP before going WIDE — the opposite of BFS.
- **Edge classification (directed):** tree edges (discovery), back edges (to an ancestor IN PROGRESS — gray), forward edges (to a finished descendant), cross edges (between finished subtrees).
- **Cycle detection:** a directed graph has a cycle ⇔ DFS finds a BACK EDGE. In undirected graphs: a cycle ⇔ a non-tree edge to an already-visited vertex that is not your immediate parent.
- **Topological sort:** order vertices by DECREASING finish time in DFS (or: insert each vertex at the FRONT of a list when it finishes). Only works on DAGs — a back edge = cycle = no topo order.

■ **WHAT GATE ASKS (patterns from 2007–2026 papers)**

- GATE pattern: "The number of connected components in this graph" → one DFS per unvisited vertex; count launches. O(V+E).
- "Which of the following is a valid topological order of this DAG?" → check every edge goes left-to-right in your candidate. The all-edges-forward test is 10 seconds per option.

```
DFS(u): // recursion = the ball of string
mark u visited; time++
for each neighbour v of u:
if v unvisited: parent[v]=u; DFS(v) // tree edge
else if v in progress: BACK EDGE -> cycle! // gray vertex
finish[u] = time++; // topo order = decreasing finish
```

5.5 Minimum Spanning Tree — cable the villages for minimum money

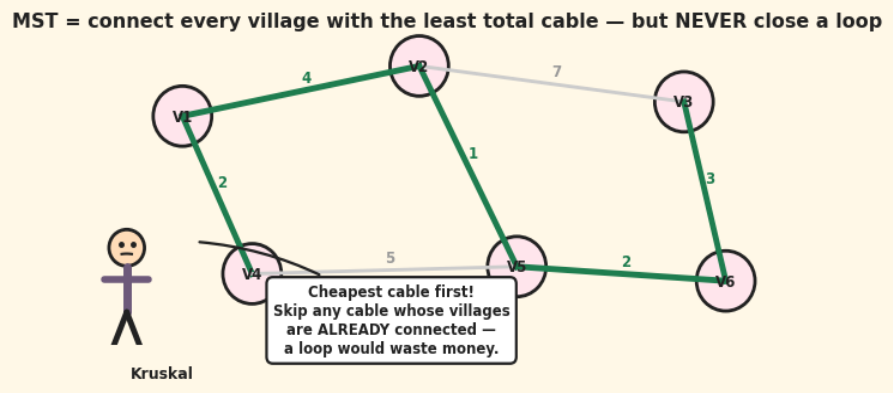


Fig 5.2 — MST keeps only cheapest connecting cables; any cable forming a loop is refused.

■ THE BOOKISH CORE (what the textbook says)

- **Cut property (the whole religion):** for any cut of the graph, the minimum-weight edge crossing the cut belongs to SOME MST. All MST algorithms are this one idea with different bookkeeping.
- **Kruskal:** sort edges by weight, add edge if its endpoints are in DIFFERENT components (union-find). O(E log E). Refuses cycle-closing edges — Fig 5.2.
- **Prim:** grow one tree from a start vertex, always add the cheapest edge leaving the tree (priority queue). O(E log V).
- **Distinct weights → unique MST.** Equal weights → possibly many; every MST still has the same total weight.

■ WHAT GATE ASKS (patterns from 2007–2026 papers)

- GATE pattern: "Which edge must be present in EVERY MST?" → an edge that is the unique minimum across SOME cut (or the unique max on no cycle — equivalently, its endpoints' only cheap connection). "May be in some MST but not all?" → tie situations.
- "The weight of the MST of this graph is…" → run Kruskal by hand on a 6–8 edge graph; add accepted weights. 2 marks, pure bookkeeping.

```
Kruskal on the Fig 5.2 weights (2,1,2,4,5,3,7):
accept V1-V4(2)? YES accept V2-V5(1)? YES accept V5-V6(2)? YES
accept V3-V6(3)? YES -> 4 edges connect 5 villages: DONE
skip V1-V2(4), V4-V5(5), V2-V3(7) <- they would close loops
MST weight = 2+1+2+3 = 8
```

5.6 Shortest paths — three algorithms, three moods

Algorithm	Edge weights	Idea	Time	When GATE uses it
BFS	All 1 (unweighted)	Levels = distance	O(V+E)	Min hops / edges count
Dijkstra	Non-negative	Greedy: settle nearest unsettled, relax its edges	O(E log V)	Trace distance table on a small graph
Bellman-Ford	Any (incl. negative)	Relax ALL edges V−1 times	O(VE)	Negative weights / detect negative cycle
DAG relax	Any (incl. negative), acyclic	Relax edges in topological order	O(V+E)	One-pass shortest path on a DAG
Floyd-Warshall	Any, no negative cycle	dp: via-k paths, k=1..V	$O(V^3)$	All-pairs, tiny graphs, transitive closure

■ WHAT GATE ASKS (patterns from 2007–2026 papers)

- **The single most repeated GATE graph task: trace Dijkstra.** Worked example (graph: A–B 4, A–C 1, C–B 2, C–D 5, B–D 1):

```
step settle A B C D (from A)
0 A 0 4 1 inf
1 C 0 3! 1 6 relax C->B: 1+2=3 < 4
2 B 0 3 1 4! relax B->D: 3+1=4 < 6
```

```
3 D FINAL: B=3, C=1, D=4
```

One settled vertex per row, each row relaxes only that vertex's outgoing edges. GATE gives 4 options of final distance arrays — the row-by-row table takes 60 seconds and cannot be wrong. Practice it until it's muscle memory.

■■ TRAP ALERT (where rank-hopefuls bleed marks)

- **Dijkstra with negative edges can produce WRONG answers** — after a vertex is settled it never reopens. GATE asks "does Dijkstra work here?" as a 1-mark trap.
- Bellman-Ford: V–1 rounds guarantee correctness IF no negative cycle. After round V–1, run one more pass — any edge still relaxing proves a negative cycle. GATE asks both facts.

■ WHAT GATE ASKS (patterns from 2007–2026 papers)

- ■ **Probable GATE 2027 (IIT-Madras style):** a counting twist — "The number of spanning trees of a complete graph on n labelled vertices is n^(n–2)" (Cayley). For n = 5 → 125. Madras papers (2011, 2019) repeatedly paired one algorithmic trace with one clean counting fact; Cayley's formula is exactly that species.

Rapid-fire quiz — 10 questions, then check answers

```
1. Edges in K_8? Directed simple graph max on 8 vertices?
2. Graph with Σ degrees = 30 → edges?
3. BFS on unweighted graph gives what?
4. DFS finds which edge type ⇒ cycle (directed graph)?
5. Valid topological order test?
6. Distinct edge weights → how many MSTs?
7. Kruskal's underlying data structure?
8. Dijkstra with a negative edge — safe?
9. Bellman-Ford: rounds? negative-cycle test?
10. Spanning trees of labelled K_5 (Cayley)?
```

■ ANSWERS (cover this box, attempt first!)

- **1.** K_8: 28. Directed: 56.
- **2.** E = 15. (Number of odd-degree vertices is always even.)
- **3.** Shortest-path distance (in hops) from the source to every vertex.
- **4.** A back edge to an in-progress (gray) vertex.
- **5.** Every edge goes forward in the order (tail before head).
- **6.** Exactly one. All MSTs (if many) share the same total weight.
- **7.** Union-Find (disjoint set) for cycle tests.
- **8.** No. Settled vertices never reopen → wrong distances possible.
- **9.** V–1 relaxation rounds; a relaxing edge in round V proves a negative cycle.
- **10.** 5^(5–2) = 125.

One-page cheat sheet — stick this on your wall

Topic	The one truth GATE keeps asking
Max edges	Undirected n(n–1)/2, directed n(n–1). Σdeg = 2E; odd-degree vertices even in count.
BFS	Queue. Levels = shortest hops on unweighted graphs. 2-color → bipartite test.
DFS	Stack/recursion. Back edge = cycle. Topo sort = decreasing finish time.
MST	Cut property rules all. Kruskal = sort + union-find. Prim = grow cheapest boundary edge.
MST uniqueness	Distinct weights → unique MST. Every MST: same total weight.
Dijkstra	Non-negative only. Settle nearest, relax neighbours. Trace = one row per settled vertex.
Bellman-Ford	Relax all edges V–1 times. Handles negatives. Extra relaxing pass = negative cycle.
DAG shortest path	Relax edges in topo order — one pass, works with negatives.
Floyd-Warshall	O(V³), all pairs, dp via-k. Also transitive closure (OR instead of min+).

Topic	The one truth GATE keeps asking
Euler vs Hamilton	Euler: all even degree. Hamilton in K_n: (n–1)!/2 — GATE 2019 (IIT-M).

Say "continue" for Chapter 6: Algorithms — sorting, hashing, master theorem, and greedy (Huffman, activity selection, optimal merge).